

QuickPool: Privacy-Preserving Ride-Sharing Service

Banashri Karmakar¹, Shyam Murthy², Arpita Patra¹ and Protik Paul³

¹ IISc Bangalore, India

² CDPG, IISc Bangalore, India

³ TU Darmstadt, Germany

Abstract. Online ride-sharing services (RSS) have become very popular owing to increased awareness of environmental concerns and also as a response to increased traffic congestion. To request a ride, users submit their locations and route information for ride matching to a service provider (SP), leading to possible privacy concerns caused by leakage of users' location data. We propose QuickPool, an efficient SP-aided RSS solution for oblivious rider-driver matching, without involving any auxiliary server. Our solution provides simultaneous multiple matching which, to the best of our knowledge, is the first one to do so. End-users, namely, riders and drivers share their route information with SP as encryptions of the ordered set of points-of-interest (PoI) of their route from their start to end locations. SP performs a zone based oblivious matching of drivers and riders, based on partial route overlap as well as proximity of start and end points. QuickPool is in the semi-honest setting, and makes use of secure multi-party computation. We provide security proof of our protocol, perform extensive testing of our implementation and show that our protocol simultaneously matches multiple drivers and riders very efficiently. We compare the performance of QuickPool with state-of-the-art works and observe a run time improvement of 1.7 - 2.2 $\times$, and communication improvement of at least 8 $\times$.

Keywords: Privacy Preservation · Ride-Sharing Service · Secure multi-party computation

1 Introduction

The European shared-mobility market is estimated to be about €70 billion as of 2022 [McK22] and projected to go to €150 billion by 2030. This projected rate of growth might not be surprising given the increasing awareness about environmental issues in the general population. Traditionally, ride-hailing service (RHS) refers to hailing a cab assisted by a central provider entity. Ride-sharing service (RSS) or carpooling, the focus of this work, is more eco-friendly than individually owned, or RHS modes of transport, as sharing a ride means fewer cars on the road leading to lesser pollution, along with the advantages of dedicated high-occupancy-vehicle lanes. In RSS, drivers are parties possessing a vehicle and willing to share a ride for a fee with riders who are willing to pay that fee for the ride, once their respective rides are matched. Based on whether a Service Provider (SP) is involved in ride-matching or not, we have the *SP-aided* or *SP-less* models, respectively. The term “matching” is described in Appendix B. There are a number of SP entities that provide RSS, namely QuickRide in India, BlaBlaCar, FlixCAR in Europe, DiDi Chuxing in China, Scoop in the US, among others.

E-mail: banashrik@iisc.ac.in (Banashri Karmakar), shyam.sm@datakaveri.org (Shyam Murthy), arpita@iisc.ac.in (Arpita Patra), protik.pmax.paul@gmail.com (Protik Paul)

Motivation. For efficient delivery of services, and for billing and statistics purposes, SPs collect information from users both at subscription time (identity, financial information) as well as for individual rides (source and destination locations). SP would keep such private information secure given the need to maintain its reputation, but there can be privacy-breach attacks [Nor20] or a malicious employee with access to such information using it for personal gains, resulting in possible financial implications on SP [The18]. Leakage of location information of riders and drivers might lead to user profiling for stalking or other malicious activities. An article published in *TheJournal* reports of a gang in Dublin, Ireland, that made use of ride-hailing apps to target taxi drivers [the19], and an article in TechCabal [Tec23], an online media company in Nigeria reported many women getting stalked by drivers after multiple failed ride-hailing requests. As is evident from these reports, individuals (drivers and riders) are vulnerable to harm when their location or other private information are available to other parties participating in the same ride-hailing app.

In light of the aforementioned issues there is a need to address the location-privacy concerns of both drivers and riders. In this work, we propose an efficient privacy-preserving ride-sharing solution (PP-RSS) wherein the locations of drivers and riders are kept private from each other as well as SP, during ride establishment. The SP-less and SP-aided models for ride-matching are briefly explained below.

SP-less model. This is a decentralized model where the SP is not needed to facilitate ride-matching, but users match their rides in some predetermined manner. In a PP-RSS solution, we expect that the locations of all parties are private and that there should not be leakage of any location information to any of the drivers (or riders) about other riders (or drivers) during ride-matching, except between the matched pairs of driver and rider at the end of the protocol. We note that a naïve SP-less model would inherently allow *all* matching drivers (or riders), that match with itself, to be known to a given rider (or driver). We refer to this as *match leakage*. This information may potentially be used by a competitor company, acting as an adversary to launch location-harvesting attacks [KMV22]. More leakages are possible when a set of riders or drivers collude with one another, but a secure protocol is expected to be secure against colluding groups of drivers, as well as against colluding groups of riders.

SP-aided model. In contrast to the SP-less model, in an SP-aided model a designated SP is involved in ride matching. As aforementioned, in a PP-RSS solution, locations of riders and drivers are hidden from all other parties, including the SP. At the end of the secure matching protocol, a rider (driver) only learns its matching driver (rider) preventing a match leakage attack. As all parties are registered with an SP, from a practical security standpoint in an SP-aided RSS setting, individuals might find it reassuring to be ride-sharing with a driver associated with a well-known SP. In the literature, there are works [YJZ⁺21, HLX⁺22] that consider a non-colluding auxiliary server along with SP, to realize efficient protocols. But in practice it maybe difficult to realize such non-colluding servers. Hence, in this work, we consider a solution involving an SP but without the involvement of any auxiliary entity.

RSS matching paradigms: In RSS, two ride matching paradigms are considered in the literature [HOS17, AHHK18, PGT⁺19]:

- Intersection based matching: If the routes of a rider and a driver have a significant overlap (higher than a threshold value), then they are considered as matched.
- End-point based matching: If the starting and ending locations of a rider and a driver are nearby (lesser than a threshold value), then they are considered as matched.

We will elaborate on both these paradigms in later sections. In this work we provide privacy-preserving versions of both paradigms.

Handling simultaneous matching requests. Given that there might be a few hundred simultaneous ride requests waiting to be serviced at a point in time, it is imperative

that ride-matching be done without much perceptible delay. Prior works in the SP-less model [HOS17, PGT⁺19, BLM⁺19, SMD16, AHHK18, XZZ⁺22] primarily focused on checking if a rider and a driver can be matched or not but did not formally consider matching multiple requests simultaneously. Therefore, we ask the following question:

Given sets of riders $\mathcal{R}$ and drivers $\mathcal{D}$, can we find maximum matches between $\mathcal{R}$ and $\mathcal{D}$ in a *privacy-preserving* manner?

To answer the above question, we first elucidate the challenges in SP-less models, as considered in prior works.

Attempt 1. Each rider independently checks against drivers until a match is found. Once a rider finds a match and the driver is available, the driver and the rider are paired. Simultaneously, the driver is marked as unavailable from all the remaining riders' lists. Each rider keeps performing the matching until it is paired with a driver. This approach requires a total $O(N^2)$ (where $|\mathcal{R}| = O(N) = |\mathcal{D}|$) matching to be performed, where each rider performs $O(N)$ sequential matching. Performing $O(N)$ sequential matching incurs significant runtime delay.

Attempt 2. Consider all riders and drivers to perform the matching protocol in parallel. Then, each rider (driver) maintains a list of drivers (riders) that are matched with itself. Now, based on some enumeration, the first rider chooses a driver from the matched list; it sends 1 to the chosen driver and 0 to the rest of the other drivers. A driver, once receiving 1 from a rider, sends 0 to all the other matched riders. Each rider and driver updates its matched list based on the received bits. This process is repeated for all the riders. Although this approach requires only $O(1)$ sequential matching, it requires $O(N^2)$ number of interactions. Furthermore, in this approach, match leakage is unavoidable.

Moreover, both attempts 1 and 2 may fail to achieve optimum matching (see Section 5).

Our solution: To handle the aforementioned challenges, we consider an SP-aided setting and use secure multi-party computation (MPC). In our construction, SP obviously checks every pair of riders and drivers for a match. The outcome of all the matching protocols is opened only to SP. This prevents match leakage. Furthermore, SP can locally perform maximum matching to output the optimum matching among the riders and the drivers. Moreover, we utilize the presence of SP to improve the single rider-driver matching. This improvement results in decreased communication as well as reduced computation which is demonstrated in our benchmarking. One of the key features of our construction is that it does not require public-key cryptography *during* the ride-matching process (excluding secure point-to-point communication channels)².

The riders and drivers in a ride-sharing service are usually mobile low-end devices, therefore it is important for a practical solution that it should not have heavy computational requirements from the riders or the drivers (public-key cryptography is, in general, computationally much more expensive than symmetric-key operations). Thus, an efficient and secure PP-RSS using lightweight symmetric-key primitives would be preferred over public-key operations. Additionally, protocols that make use of quantum-safe primitives would definitely be preferable to ones that are deemed quantum unsafe.

We ask the question whether we can have an efficient PP-RSS solution that works in an SP-aided model using only lightweight symmetric key quantum safe primitives, without using any auxiliary server, and provides only the matching specific information to just the selected driver-rider pair while maximizing the number of simultaneous matches. In this work, we answer that question in the affirmative.

²Public-key cryptography is used only to setup a secure point-to-point channel which can be instantiated using a quantum-safe key exchange protocol

1.1 Related Works

There is a long list of works that look at the various aspects of PP-RSS. In this section, we list a few works to illustrate how our work addresses some of the drawbacks present in earlier works, and is summarized in Table 1. For a more complete list of related works, please refer to Appendix A.

One of the early PP-RSS protocols, PrivatePool, by Hallgren et al. [HOS17] obviously

Table 1: Prior works listing one or more feature specific drawbacks of the protocol

	Resilient to match leakage	No auxiliary server	Quantum-safe ² / symmetric-key
[HOS17], [PGT ⁺ 19]	✗	✓	✗
[BLM ⁺ 19], [SMDF16]	✗	✓	✓
[LWD23], [ZHQ25]	✓	✗	✗
[AHHK18], [XZZ ⁺ 22]	✗	✓	✗
QuickPool	✓	✓	✓

matches riders based on proximity matching of end points as well as overlaps in riders' route trajectories. TOPPool [PGT⁺19] is a follow-up work by Pagnin et al. that incorporates the time of travel for ride matching. It used private set intersection for route segment matching and used an additive homomorphic encryption scheme along with a privacy-preserving matching method to aid in oblivious end-point matching. The works of Baza et al. [BLM⁺19] and Sanchez et al. [SMDF16] are both decentralized models, with the former using a public blockchain while the latter used peer trust enforcement using a distributed reputation management protocol. The work by Li et al. [LWD23] used road network embedding (RNE) to represent the road network and made use of additive homomorphic encryption (AHE) to securely compute distances between riders, and used an auxiliary non-colluding server for decryption. A recent work by Zhang et al. [ZHQ25] proposed a PP-RSS solution in a malicious adversarial setting and considered path intersection of multiple simultaneous riders. Their protocol internally uses OPRF which has computation overhead, and also makes use of a non-colluding auxiliary *trusted authority* responsible for system initialization and registration. SRide [AHHK18] by Aïvodji et al. made use of secure multi-party computation (MPC) together with SHE to obviously match riders and drivers, and provided results for intersection based matching. In their solution, a rider gets to know all matched drivers which can lead to potential location-harvesting attacks by colluding riders, for example, [MV22]. A work called TAROT by Xu et al. [XZZ⁺22] made use of Goldwasser-Micali (GM) public-key encryption scheme to match end-users using reverse minhash-based Jaccard similarity. Here again, the end-user decrypts all the encrypted matching results using her public key, which can potentially lead to adversarial attacks.

1.2 Our Contributions and Technical Overview

To the best of our knowledge, most of the works either rely on heavy public-key primitives [SMDF16, PGT⁺19, HOS17, YZY⁺21] or make a strong assumption of the existence of additional (non-colluding) auxiliary servers [HLX⁺22, YJZ⁺21, YZY⁺21, XZZ⁺22]. As observed in Section 1.1, [AHHK18, XZZ⁺22] may have potential privacy concerns, which we aim to circumvent by considering the SP-aided model. We also consider the real-world scenario where multiple rider-driver match requests are required to be processed simultaneously to obtain a maximal matching, and we believe that we are the first to do so. Our work can be used as a building block to build a fully deployable privacy-preserving RSS application, by adding features like billing, feedback rating etc. So, the focus of this

work is to design a practical oblivious ride-matching solution with low-latency. Also, the solution must be extremely fast, generate high throughput (number of matches in unit time), and be lightweight to support end-users with low-end mobile devices. We choose our system model and design the protocols keeping the above goals in mind. We provide a summary of the contributions and a high-level overview of this work.

- **System model.** In this work, we consider PP-RSS in the SP-aided model, where the location privacy of end-users is preserved from the SP, considering it as semi-honest, and as well as other users (adversarial model is described in Section 2.1). With the SP aiding in oblivious matching, we address the aforementioned concerns, and our construction achieves the following properties.

- *Symmetric key versus public key primitives.* Our construction relies only on symmetric key primitives (PRG, PRF and secret sharing) and does not rely on public key primitives during ride-matching, except during initialization to setup secure communication channel. Thus our work is plausibly post-quantum secure and also achieves better efficiency and suitable for low end mobile devices.

- *Local maximum matching.* In our construction, only SP obtains the output corresponding to every matched pair of riders and drivers and can execute any local matching algorithm such that the maximum number of riders and drivers is matched, which is not the case in an SP-less model.

- *Strong security guarantee.* In some of the prior works [AHHK18, XZZ⁺22], a rider learns about all drivers that satisfy the matching criteria with itself, thus weakening the security. In our construction, a rider (or driver) learns about only one driver (or rider) with whom it is matched.

- **Intersection based matching.** Using an approach similar to [PGT⁺19], we consider riders with routes selected by their mobile apps and declare a match based on cardinality of their intersection sets. For this, we use private set intersection (PSI) using PRFs which are cheaper than PSI using oblivious PRFs (OPRF) used by [PGT⁺19]. SP locally executes the PSI to find the cardinality of the intersection of riders' routes. We achieve significant speed-up compared to [PGT⁺19] e.g., for a single rider and driver, in [PGT⁺19], each user incurs $O(n \log n)$ for both communication and computation costs while we incur only $O(n)$, where n is the size of the route. We provide the details in Section 3.1.

- **End-point based matching.** We support end-point based matching, where if the starting and ending locations of the rider and the driver are 'nearby' then they are a match, and use Euclidean distance as the metric. We resort to techniques of multiparty computation (MPC) [BOGW88, CCD88, PSSY21] where the inputs (namely the co-ordinates) are secret shared among the SP, driver and rider parties. We use MPC with 3 parties, secure against one semi-honest adversary, for distance computation and function secret sharing [BCG⁺21, ICÖ22] for comparisons. Our protocol achieves a quadratic improvement over [PGT⁺19] in both communication and computation costs. We provide the details in Section 3.2.

- **Simultaneous matching.** Most of the prior works focus on matching a single rider-driver. However, while processing multiple requests, it may be possible that some users get multiple matches whereas some others get none at all leading to starvation of a set of users. To mitigate this in Quick Pool, SP executes a maximum bipartite matching to maximize the number of matches. We elaborate more on this in Section 5.

- **Benchmarks.** We implement both the intersection and end-point matching algorithms, and compare the performance with the state-of-the-art protocols from TOPPool [PGT⁺19]. We observe that our intersection based matching protocol achieves almost $1.7 - 2.2\times$ speedup and requires at least $8\times$ less communication as compared to O-PrivatePool. On the other hand, our end-point based matching protocol has $4.5\times$ less communication cost

and is $1.10\times$ faster compared to TOPPool, even for a small matching threshold value. Furthermore, this gap increases with the matching threshold value. For the end-point based matching protocol, we can complete the matching of up to 40 riders and 40 drivers within 37 seconds, whereas, for the intersection based matching protocol, within the same time, we can handle up to 100 riders and 100 drivers. However, the benefit of the end-point matching protocol is that it requires each end-user to transmit significantly less data compared to SP. For instance, in scenarios with up to 100 riders and 100 drivers, each end-user needs to send only $\sim 6.25\text{KB}$. QuickPool achieves a throughput of ~ 27 matches/min for end-point based matching and ~ 746 matches/min for intersection based matching, both in a scenario involving 100 riders and 100 drivers. Further details are provided in Section 6.

- **Scalability considerations.** In times of peak-traffic, it is quite likely that the number of drivers and riders can each be considerably high. In order to have a scalable solution, we propose to execute our matching in epochs of σ seconds, where σ is configurable by the SP based on real-time considerations like public events, peak-hour traffic etc. Additionally, SP will also be able to tune a parameter β , the batch size of driver and rider requests on which the matching algorithms are executed. Every σ seconds, β number requests are selected (in the order of arrival) for matching. At the end of the epoch, any unmatched requests are pooled with new requests and the matching procedure is repeated. We note here that executing the matching algorithm in batches might not give the global optimum maximum matching, but we adopt this method to be able to handle load surges gracefully.

Organization. In Section 2, we give with a brief discussion about the assumptions and threat models considered in QuickPool. Section 3 provides the technical details of our constructions to match a single pair of riders and drivers. In Section 4, we provide the formal security proofs of our protocols. Matching multiple riders and drivers simultaneously is discussed in Section 5. Section 6 provides the benchmarking results. Finally, in Section 7 we conclude with a set of features for a practical and comprehensive PP-RSS solution.

2 Assumptions and Threat Model

2.1 System and Adversarial Model

We consider an online Ride-Sharing System (RSS) involving an SP. The SP receives encrypted information from each of the end parties, facilitates ride-matching, and informs the successful parties. The details of the matching algorithms are described in detail in Sections 3.1 and 3.2.

We consider a semi-honest setting where all parties follow the protocol. We allow collusion among drivers and also collusion among riders. The rationale behind opting for semi-honest over malicious security is the following. A malicious SP can only alter the output; i.e., it can misidentify a matched pair as unmatched or vice-versa. This would lead to conflict among users, resulting in a bad reputation for SP. On the other hand, a malicious user may provide incorrect inputs or communicate incorrect intermediate values, which do not lead to privacy breaches. Hence, we assume semi-honest SP and users. However, using techniques from [KKP⁺24], we can achieve security against malicious users by using authenticated secret sharing with increased cost of communication and computation. Further, we assume that SP does not collude with drivers or riders, and drivers do not collude with riders. However, each party is interested to know more about the other parties than what is ordinarily available and computable from the protocol transcription. In both the matching methods, the location information of each driver and rider is expected to be kept oblivious from SP and all other participating parties. After a successful match, only the two matched parties get to know each other's locations and

none of the other participating drivers or riders learn anything.

2.2 Assumptions and Limitations

As in prior works, secure communication channels are assumed to be present between the end-users, and between SP and end-users. In this work, riders and drivers make use of their mobile devices for ride-matching requests via the mobile apps, provided by the SP at the time of subscription. The apps are responsible for keeping an updated view of the road network topology and points-of-interest (PoIs), (like landmarks, road-intersections), of the area of operation. Given the source and destination points of the end-users, the apps are expected to obtain an optimum route consisting of PoIs and follow the protocol as described in the paper. Any security breach from the point of view of the apps is out of the scope of this work. In our protocol, SP provides match results of both types of matching and the mobile apps should have the capability to convey the same to the user and solicit user input on which of the two results to use for final matching.

3 QuickPool Single Driver-Rider Matching

During ride-matching in RSS, SP tries to match between a rider R with a driver D . R and D are termed “matched” if certain criteria are met. These criteria ensure that R or D do not deviate from their intended route more than their acceptable deviation threshold. Considering such constraints, as aforementioned, primarily two paradigms are considered for matching, namely, intersection based matching and end-point based matching. In this work, we consider both approaches.

In this section, we describe how a single pair of rider-driver matching is performed. In Section 5, we will see how to match more than one pair simultaneously. In summary, every rider-driver pair together with the SP executes the matching algorithms (Section 3.1 and Section 3.2), and SP obtains the output for every possible pair. The algorithm in Section 5 uses the output of Section 3 to execute a local matching algorithm to output multiple matching pairs. It then returns the matching result to each successful rider and driver.

We introduce a few terms to help follow our constructions. For more details please refer to Appendix B. Our constructions rely on secure multi-party computation (MPC) [BOGW88, CCD88] that allows multiple parties to jointly and securely compute a common function. We make use of distributed comparison function (DCF) [BCG⁺21] for secure comparison, and private set intersection (PSI) for set intersection. Rider-driver (R)-(D) matches are done within suitable size grids demarcated by SP for ease of operation, and locations of R and D are shared with SP as encryptions of their closest point of interest (PoI), like a landmark or road intersection. The path from source to destination of each user, R or D , termed route, is an ordered set of PoIs along the path as determined by the RSS app on users’ mobile device. As assumed in prior works, a secure communication channel is present between each user and SP, as well as between R and D . Appendix B explains key exchange as well as common key creation. Section 2.1 provides details on the adversarial model.

3.1 Intersection Based Matching

When a rider R raises a request for matching, it looks for a driver which has an aligning route with itself. Similarly, a driver D would prefer a rider with whom its route matches. This notion of preference is formalized in the form of intersection based matching, where R and D provide their routes, and if there is a significant overlap in the route (as specified by a threshold), they are matched. A route, τ , can be modeled as a path in a graph $G = (V, E)$ where G represents the road network of the operational area. V is the set of PoI identifiers,

and $(u, v) \in E$ be an edge if u is ‘reachable’ from v . For simplicity, we assume that for a given source and destination pair, there is a unique designated route. Furthermore, a route is represented as a sequence of edges, that is $\tau = \{(v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)\}$. Along with the route, a user also provides the matching threshold, which can be a public value. However, it is required that the routes of the users remain hidden. Below, we will first discuss how to check if a rider and a driver are matched in a privacy-preserving manner. Let R and D be a rider and a driver respectively with routes $\tau_R = \{(v_1^{(R)}, v_2^{(R)}), \dots, (v_{m-1}^{(R)}, v_m^{(R)})\}$ and $\tau_D = \{(v_1^{(D)}, v_2^{(D)}), \dots, (v_{n-1}^{(D)}, v_n^{(D)})\}$. Inputs from R are τ_R (private) and the threshold t_R (public), similarly, D ’s inputs are τ_D and t_D . R and D are matched if $|\tau_R \cap \tau_D| \geq \max\{t_R, t_D\}$.

PrivatePool [HOS17] provides a solution to the above problem by using threshold private set intersection (TPSI). However, the construction of TPSI has significant performance overhead (Sec. §1, pp. 2, [PGT⁺19]). TOPPool [PGT⁺19] addresses this issue by moving from TPSI to standard PSI without incurring any additional overhead. However, their PSI construction relies on public-key cryptographic primitives such as OPRF, which has significant computational overhead. In this work, we propose a solution without relying on any public-key cryptographic primitive, thus achieving an extremely fast and lightweight solution.

We consider SP as one of the computational parties (without input) that facilitates the computation. On a very high level, our approach is the following. R and D establish a common key among themselves which is unknown to SP. They use the key to evaluate a PRF (not the costly OPRF) locally on all segment end-points (edges) of their respective routes. They send the evaluated values to SP. SP obtains the cardinality of the intersection locally, if the threshold condition is met, it sets R and D as matched. Observe that the values received by SP appear completely random as they are evaluations of PRF. We elaborate further on the construction below.

R and D execute Π_{KeySetup} to establish a common key k (Appendix B, Fig. 12). Let $F : \{0, 1\}^\ell \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ be a secure PRF. R and D use key k to evaluate F on their private inputs, that is,

R computes $y_j^{(R)} = F(k, v_j^{(R)} || v_{j+1}^{(R)}), \forall j \in [m-1]$, D computes $y_j^{(D)} = F(k, v_j^{(D)} || v_{j+1}^{(D)}), \forall j \in [n-1]$.

Protocol Π_{Int}

Input:

- R ’s private input: $\tau_R = \{(v_1^{(R)}, v_2^{(R)}), \dots, (v_{m-1}^{(R)}, v_m^{(R)})\}$ and public input: t_R .
- D ’s private input: $\tau_D = \{(v_1^{(D)}, v_2^{(D)}), \dots, (v_{n-1}^{(D)}, v_n^{(D)})\}$ and public input: t_D .
- SP does not have any input in the protocol.
- Public parameter L , denotes maximum route length.

Output: 1 if $|\tau_R \cap \tau_D| \geq \max\{t_R, t_D\}$, otherwise 0

Protocol:

- R and D perform key setup by executing $\Pi_{\text{KeySetup}}(R, D, SP)$ (Fig. 12), and agree on a common key k .
- R computes $y_j^{(R)} = F(k, v_j^{(R)} || v_{j+1}^{(R)})$ for all $j \in [m-1]$.
- R samples $L - m$ random values, $y_m^{(R)}, \dots, y_{L-1}^{(R)}$ from $\{0, 1\}^\ell$. It sets $Y_R = \{y_1^{(R)}, \dots, y_{m-1}^{(R)}, y_m^{(R)}, \dots, y_{L-1}^{(R)}\}$.
- D computes $y_j^{(D)} = F(k, v_j^{(D)} || v_{j+1}^{(D)})$ for all $j \in [n-1]$. Further, it sets $Y_D = \{y_1^{(D)}, \dots, y_{n-1}^{(D)}\}$.
- D samples $L - n$ random values, $y_n^{(D)}, \dots, y_{L-1}^{(D)}$ from $\{0, 1\}^\ell$. It sets $Y_D = \{y_1^{(D)}, \dots, y_{n-1}^{(D)}, y_n^{(D)}, \dots, y_{L-1}^{(D)}\}$.
- R and D send (Y_R, t_R) and (Y_D, t_D) respectively to SP.

- SP computes $d = |Y_R \cap Y_D|$ and sets **Output** = 1 if $d \geq \max\{t_R, t_D\}$ otherwise sets **Output** = 0.

Figure 1: Intersection based ride-sharing matching

R and D, respectively, send their threshold values t_R and t_D , and Y_R and Y_D to SP, where $Y_R = \{y_1^{(R)}, \dots, y_{m-1}^{(R)}\}$ and $Y_D = \{y_1^{(D)}, \dots, y_{n-1}^{(D)}\}$. However, this leaks the route lengths of rider and driver to SP. To hide the route length, we consider a public parameter, L that indicates the maximum route length. R and D locally sample $L - m$ and $L - n$ values from $\{0, 1\}^\ell$ respectively. Note that these sampled values are random, with negligible probability they will contribute to the matching. Thus R sends $Y_R = \{y_1^{(R)}, \dots, y_{m-1}^{(R)}, y_m^{(R)}, \dots, y_{L-1}^{(R)}\}$ and D sends $Y_D = \{y_1^{(D)}, \dots, y_{n-1}^{(D)}, y_n^{(D)}, \dots, y_{L-1}^{(D)}\}$ to SP, where R samples $y_m^{(R)}, \dots, y_{L-1}^{(R)}$ and D samples $y_n^{(D)}, \dots, y_{L-1}^{(D)}$ locally.

Note that, since the SP receives the PRF evaluations of the private inputs it does not learn any information about the underlying values. However, it can compute the cardinality of the intersection on the evaluated values. Furthermore, SP checks if the cardinality of the intersection is more than the provided threshold values, namely t_R and t_D . If the matching criteria are met, SP sets R and D as a match. Observe that, there are no messages exchanged between the rider and the driver that depend on their private inputs, hence privacy is guaranteed. Below, we describe the protocol Π_{Int} (Fig. 1) The protocol Π_{Int} requires only 1 round of communication and $m + n$ elements. In this protocol, the computations on the riders' and drivers' sides are extremely lightweight (PRF evaluation on their local input), whereas the rest of the matching is performed by the service provider on PRF outputs.

3.2 End-point Based Matching

Orthogonal to the intersection based matching, end-point based matching algorithm is well-studied in the literature. In this matching algorithm, if a driver and rider pair have the start positions as well as the end positions close by, then they are a match. As aforementioned, to further preserve anonymity we use the PoI closest to the start or end location to identify the respective location, and refer to the UTM coordinates of the respective PoIs as the start and end locations. We compute the Euclidean distance between the rider and the driver, and if it is lesser than a predetermined threshold value then we consider them as a match. To compute the same, we rely upon MPC and FSS techniques.

Matching a pair. A rider R and a driver D are considered to be matched if they satisfy the following criteria.

1. The distance between the start locations of R and D is $\leq \rho_s$, where ρ_s is the threshold start distance.
2. The distance between the end locations of R and D is $\leq \rho_e$, where ρ_e is the threshold end distance.

In TOPPool [PGT⁺19], an additive homomorphic encryption scheme (Paillier's scheme [Pai99]) is used to encrypt the locations of the rides, an oblivious end-point matching algorithm with secure comparison is run to determine if any two riders can be matched. In our method, we securely compute the Euclidean distance, respectively, between the start locations, the end locations and use the respective threshold values to determine a successful match.

We perform the check in two steps. In the first step, R, D, and SP execute a 3-party MPC protocol, with 1 semi-honest corruption, to compute the square of the Euclidean distances of the corresponding start and end locations. This is followed by two secure comparison protocols.

3-party computation of Euclidean distance (ED). To check if R and D are a match or not, we compute the square of the Euclidean distance between the start locations of the R and D. Then we compare it with the square of the threshold value. We do the same for the end locations as well. If both conditions are met, then they are considered as a match. Below we describe how we compute the Euclidean distance between two secret locations (we describe the same for the start points. For the end points, it will follow similarly).

The riders and the drivers have their input as the start location and the end location. As aforementioned, a location is represented by its closest PoI and its UTM coordinates. Let R's inputs be $rLoc_s = (rx_s, ry_s)$ and $rLoc_e = (rx_e, ry_e)$, similarly, D's inputs be $dLoc_s = (dx_s, dy_s)$ and $dLoc_e = (dx_e, dy_e)$. We let $ED(u, v)$ to be the Euclidean distance between two points u and v . We need to obviously compute $SDS = [ED(rLoc_s, dLoc_s)]^2$ and $EDS = [ED(rLoc_e, dLoc_e)]^2$.

Note that, for the above computation, one part of the input is held by R and the other part is available with D. We aim to compute the SDS and EDS without revealing anyone's input to any party other than the input holder. For this, we rely on the secure multiparty computation (MPC) tool, where an input holder splits the input into multiple parts in such a way that each of the parts does not contain any information about the input. In other words, each of the parts looks random and the input can be retrieved only if all the parts are put together. Splitting the input in such a way is commonly known as secret sharing. The computation of the desired function happens in two alternate stages, local computation and interaction. In the first step, R and D prepare for the secret sharing of their inputs. In the communication phase, they exchange the corresponding shares of the inputs. In the later phase, to compute the function, R and D perform local computation followed by another round of interaction to complete the distributed computation. Below we elaborate more on the local computation and communication.

Secret sharing semantics. In this work, we will use two types of secret sharing.

- *Additive secret sharing.* Let s be a secret (held by one of the two parties) and s is additively shared between R and D. $[s]$ denotes the shares of s . $[s]_r$ and $[s]_d$ be the shares of R and D respectively. Note that, $[s]_d + [s]_r = s$. Furthermore, $[s]_r$ and $[s]_d$ are random values, therefore individually these values do not reveal any information about s .
- *Augmented additive sharing.* As before, consider s to be a secret, and $\llbracket s \rrbracket$ represents the augmented sharing of s . The augmented sharing of s consists of two components: a random mask (δ_s) and a masked value ($m_s = s + \delta_s$). In the augmented sharing, both R and D hold the masked value m_s ; however, the mask (δ_s) is additively shared among the rider and the driver.

Note that, both the secret sharing mentioned above have the homomorphic property, that is, $[c_1x + c_2y] = c_1[x] + c_2[y]$ and $\llbracket c_1x + c_2y \rrbracket = c_1\llbracket x \rrbracket + c_2\llbracket y \rrbracket$ where c_1 and c_2 are publicly known values. Here the parties with their respective secrets perform the operations and obtain the shares of the computed values.

Sampling common random values. We will use the common keys, obtained from $\Pi_{KeySetup}$ (Fig. 12), to sample common random values. This will be required for the input sharing as well as for the computation. If more than one party is involved in sampling a random value, in that case, they will use the common key (as discussed in the above paragraph) to evaluate a PRF on an agreed value (e.g, a counter, ctr , which is incremented after each evaluation). For example, consider R and SP wish to sample a common random value, v , then both SP and R evaluate $PRF(k_r, ctr) = v$.

Input sharing. R has four input values rx_s, ry_s, rx_e and ry_e . We aim to achieve augmented additive sharing of these values. To share these values, R and SP sample random values $\delta_{rx_s}, \delta_{ry_s}, \delta_{rx_e}$ and δ_{ry_e} using the key k_r , these are the masks of the respective values. It is required to generate additive sharing of these values. For that, SP and R sample $[\delta_{rx_s}]_r, [\delta_{ry_s}]_r, [\delta_{rx_e}]_r, [\delta_{ry_e}]_r$ using the key k_r . SP computes $[\delta_{rx_s}]_d = \delta_{rx_s} - [\delta_{rx_s}]_r$ and sends it to D. SP does the same for $[\delta_{ry_s}]_d, [\delta_{rx_e}]_d, [\delta_{ry_e}]_d$. Then, R locally computes $m_{rx_s} = rx_s + \delta_{rx_s}$.

Similarly, R computes m_{ry_s}, m_{rx_e} and m_{ry_e} . R sends $m_{rx_s}, m_{ry_s}, m_{rx_e}$ and m_{ry_e} to D. To share D's input, D, R and SP follow the similar steps.

Computation of SDS and RDS by R and D. Using the homomorphic property of the $\llbracket \cdot \rrbracket$ -sharing scheme, R and D compute $\llbracket rx_s \rrbracket - \llbracket dx_s \rrbracket = (m_{rx_s} - m_{dx_s}, [\delta_{rx_s}] - [\delta_{dx_s}])$. Similarly, R and D compute $\llbracket ry_s \rrbracket - \llbracket dy_s \rrbracket$, $\llbracket rx_e \rrbracket - \llbracket dx_e \rrbracket$ and $\llbracket ry_e \rrbracket - \llbracket dy_e \rrbracket$. Given vectors $\mathbf{x}$ and $\mathbf{y}$, their dot product $\mathbf{z}$ is shown as $\langle \mathbf{x}, \mathbf{y} \rangle$. Since

$$\text{SDS} = \langle (rx_s - dx_s, ry_s - dy_s), (rx_s - dx_s, ry_s - dy_s) \rangle, \quad \text{EDS} = \langle (rx_e - dx_e, ry_e - dy_e), (rx_e - dx_e, ry_e - dy_e) \rangle,$$

we will describe how to compute $\mathbf{z}$ such that each component is an augmented additive share. Note that

$$\llbracket \mathbf{x} \rrbracket = (\llbracket x_1 \rrbracket, \llbracket x_2 \rrbracket) = ((m_{x_1}, [\delta_{x_1}]), (m_{x_2}, [\delta_{x_2}])), \quad \llbracket \mathbf{y} \rrbracket = (\llbracket y_1 \rrbracket, \llbracket y_2 \rrbracket) = ((m_{y_1}, [\delta_{y_1}]), (m_{y_2}, [\delta_{y_2}])).$$

Here $\delta_{x_1}, \delta_{x_2}, \delta_{y_1}, \delta_{y_2}$ are additively shared among R and D. However, SP holds these values in clear. To compute $\langle \mathbf{x}, \mathbf{y} \rangle$, SP prepares the following: $\delta_{xy} = \delta_{x_1} \cdot \delta_{y_1} + \delta_{x_2} \cdot \delta_{y_2}$ and generates additive sharing of it. To generate the additive sharing of δ_{xy} , SP and R sample a common random value $[\delta_{xy}]_r$ using the common key k_r . Then SP computes $[\delta_{xy}]_d = \delta_{xy} - [\delta_{xy}]_r$ and sends it to D. Furthermore, SP, R and D prepare $[\cdot]$ -sharing of a random value, say δ_z . For that SP and R sample a random value $[\delta_z]_r$ using their common key k_r and SP and D also sample another random value $[\delta_z]_d$ using their common key k_d . SP sets $\delta_z = [\delta_z]_r + [\delta_z]_d$. Observe that, R and D hold additive sharing of δ_z , the mask of $\langle \mathbf{x}, \mathbf{y} \rangle$. However, to maintain the sharing semantics, they are required to compute the masked value m_z . Note that

$$\begin{aligned} \mathbf{z} &= x_1 y_1 + x_2 y_2 \\ \Rightarrow m_z &= (m_{x_1} - \delta_{x_1})(m_{y_1} - \delta_{y_1}) + (m_{x_2} - \delta_{x_2})(m_{y_2} - \delta_{y_2}) + \delta_z \\ \Rightarrow m_z &= m_{x_1} m_{y_1} + m_{x_2} m_{y_2} - m_{x_1} \delta_{y_1} - m_{y_1} \delta_{x_1} - m_{x_2} \delta_{y_2} - m_{y_2} \delta_{x_2} + \delta_{x_1} \delta_{y_1} + \delta_{x_2} \delta_{y_2} + \delta_z \end{aligned}$$

Let $\delta_{xy} = \delta_{x_1} \delta_{y_1} + \delta_{x_2} \delta_{y_2}$. R and D respectively compute

$$[m_z]_r = m_{x_1} m_{y_1} + m_{x_2} m_{y_2} - m_{x_1} [\delta_{y_1}]_r - m_{y_1} [\delta_{x_1}]_r - m_{x_2} [\delta_{y_2}]_r - m_{y_2} [\delta_{x_2}]_r + [\delta_{xy}]_r + [\delta_z]_r \quad (1)$$

$$[m_z]_d = -m_{x_1} [\delta_{y_1}]_d - m_{y_1} [\delta_{x_1}]_d - m_{x_2} [\delta_{y_2}]_d - m_{y_2} [\delta_{x_2}]_d + [\delta_{xy}]_d + [\delta_z]_d. \quad (2)$$

To complete the computation, R and D exchange their shares of m_z with each other. R and D both locally compute $m_z = [m_z]_r + [m_z]_d$. Therefore, $\llbracket \mathbf{z} \rrbracket = (m_z, [\delta_z])$.

Output. R, D and SP complete this phase of the computation by outputting $\llbracket \text{SDS} \rrbracket$ and $\llbracket \text{EDS} \rrbracket$ where SP holds δ_{SDS} and δ_{EDS} and R and D hold m_{SDS} and m_{EDS} .

Performing the comparison: To decide if R and D are matched or not, it is required to check if $\text{SDS} \leq \rho_s^2$ & $\text{EDS} \leq \rho_e^2$ are satisfied. In the literature, there are different ways of performing comparison. Considering bit representations of decimal values and the most significant bit (msb) as the sign bit, extracting the msb provides the output of the comparison with 0. This approach is widely used in various prior works [KKP⁺24, CDH10, PSSY21, HKK⁺22]. It requires evaluating a boolean circuit of depth $\log_2(k)$ (later works [KKPRG22, KKP⁺24, PSSY21] optimized it evaluating a circuit with depth $\log_4(k)$), where the computation is performed over $\mathbb{Z}_{2^k}$. This approach incurs more interaction among the parties. Another line of works considers function secret sharing based approach [BGI15, BCG⁺21, ICÖ22], where to evaluate a comparison, a distributed comparison function (DCF) is evaluated, which is represented by a key. The keyholder locally evaluates the DCF using its key and obtains a random sharing of the output. However, generating the keys for the DCF is heavier when it is generated using MPC. Many prior works [ICÖ22] bypass this challenge by considering an additional helper party other than the evaluators. Fortunately, in our case, the helper party is available naturally in the form of SP. In our construction, SP generates the DCF keys for R and D. Upon receiving the keys, R and D perform the DCF evaluation locally and obtain an additive sharing of the output. Below, we elaborate further on the execution of the comparison

protocol using DCF. Consider a function $f_\alpha(x)$, parameterized by α , such that $f_\alpha(x) = 1$ if $x \leq \alpha$, 0 otherwise.

The Gen algorithm of DCF takes α as an input and outputs K_0 and K_1 . The helper party (SP) executes the Gen algorithm locally and sends K_0, K_1 to the evaluators. The evaluators (R and D) hold additive shares of the secret x and α . They reconstruct $x + \alpha$ by exchanging the shares of $x + \alpha$, which is obtained by locally applying the homomorphic property of the additive secret sharing. The evaluators perform the Eval algorithm locally to obtain the sharing of the comparison output. In this work, since SP holds δ_{SDS} and δ_{EDS} where these values are already additively shared with R and D, δ_{SDS} and δ_{EDS} are considered as the α of the corresponding comparison protocols. SP generates keys by providing δ_{SDS} and δ_{EDS} as input to the Gen algorithm. R and D upon receiving the keys perform the Eval on $m_{SDS} - \rho_s^2$ and $m_{EDS} - \rho_e^2$. R and D get the sharing of the output of the comparison protocol. They send the shares to SP, who reconstructs the output, and if both conditions are met (that is, both the outputs are 1), it sets R and D as matched. We describe the Π_{EndPoint} (Fig. 2) protocol below. The communication cost of Π_{EndPoint} is 193 elements and requires 4 rounds of interaction.

We provide the formal security proof in Section 4, where we design simulators corresponding to the adversary corrupting either SP or R(D). Then we discuss how the simulated view is indistinguishable from the real view of the adversary.

Protocol $\Pi_{\text{EndPoint}}(R, D)$

Input:

- R's input rx_s, ry_s, rx_e and ry_e .
- D's input dx_s, dy_s, dx_e and dy_e .
- SP does not have any input in the protocol.
- Public input ρ_s and ρ_e .

Output: 1 if $SDS \leq \rho_s^2$ & $EDS \leq \rho_e^2$, 0 otherwise, where

$SDS = \langle (rx_s - dx_s, ry_s - dy_s), (rx_s - dx_s, ry_s - dy_s) \rangle$

$EDS = \langle (rx_e - dx_e, ry_e - dy_e), (rx_e - dx_e, ry_e - dy_e) \rangle$.

Protocol:

- *Input sharing.* For an input v , consider P (either R or D) to be the input owner.
 - SP and P sample a random mask δ_v . SP and R sample random shares of δ_v , $[\delta_v]_R$. SP computes $[\delta_v]_D = \delta_v - [\delta_v]_R$ and sends it to D.
 - P computes $m_v = v + \delta_v$ and sends it to the other party, that if $P = R$, it sends it to the D and vice versa.
 - R and D obtain $\llbracket v \rrbracket = (m_v, [\delta])$
- *Computation of SDS and EDS.*
 - *Local computation.* Parties compute $\llbracket rx_s - dx_s \rrbracket = \llbracket rx_s \rrbracket - \llbracket dx_s \rrbracket$ and $\llbracket ry_s - dy_s \rrbracket = \llbracket ry_s \rrbracket - \llbracket dy_s \rrbracket$. Similarly, they compute $\llbracket rx_e - dx_e \rrbracket$ and $\llbracket ry_e - dy_e \rrbracket$.
 - *Dot product.* Let $v_1 = rx_s - dx_s, v_2 = ry_s - dy_s, v_3 = rx_e - dx_e, v_4 = ry_e - dy_e$.
 - SP computes $\delta_{12} = \delta_{v_1}^2 + \delta_{v_2}^2$ and $\delta_{34} = \delta_{v_3}^2 + \delta_{v_4}^2$. SP and R sample $[\delta_{12}]_R$ and $[\delta_{34}]_R$. Then, SP computes $[\delta_{12}]_D = \delta_{12} - [\delta_{12}]_R$ and $[\delta_{34}]_D = \delta_{34} - [\delta_{34}]_R$ and sends $[\delta_{12}]_D$ and $[\delta_{34}]_D$ to D. SP and R sample random values $[\delta_{SDS}]_R$ and $[\delta_{EDS}]_R$. Similarly, SP and D sample random values $[\delta_{SDS}]_D$ and $[\delta_{EDS}]_D$. SP sets $\delta_{SDS} = [\delta_{SDS}]_R + [\delta_{SDS}]_D$ and $\delta_{EDS} = [\delta_{EDS}]_R + [\delta_{EDS}]_D$.
 - R and D compute m_{SDS} and m_{EDS} following (1) and (2) respectively.
- *Comparison.*
 - SP executes DCF Gen algorithm with input δ_{SDS} and δ_{EDS} and generates the keys $K_{SDS}^r, K_{SDS}^d, K_{EDS}^r$ and K_{EDS}^d . SP sends K_{SDS}^r, K_{EDS}^r to R and K_{SDS}^d, K_{EDS}^d to D.
 - R performs DCF Eval algorithm with K_{SDS}^r and K_{EDS}^r on input $m_{SDS} - \rho_s^2$ and $m_{EDS} - \rho_e^2$.

- D performs DCF Eval algorithm with K_{SDS}^d and K_{EDS}^d on input $\text{m}_{\text{SDS}} - \rho_s^2$ and $\text{m}_{\text{EDS}} - \rho_e^2$.
- R and D send their outputs to SP.
- *Output.* SP reconstructs the outputs. If both the outputs are 1, SP outputs 1, 0 otherwise.

Figure 2: End-point based ride-sharing matching

4 Security Proofs

4.1 Security Proof of Π_{Int}

In this section, we discuss proof of security of our constructions. We claim that the security of our protocols will be preserved even if one of the three parties (SP, R, D) is corrupt (semi-honest). We establish this by showing that there is a simulator (probabilistic polynomial time algorithm) that generates an indistinguishable view of the corrupt parties without participating in the protocol. Due to the asymmetric nature of our protocol, we will first provide the simulator $\mathcal{S}_{SP}$ for a corrupt SP, and then we will argue why the simulated view is indistinguishable from the view of the corrupt party.

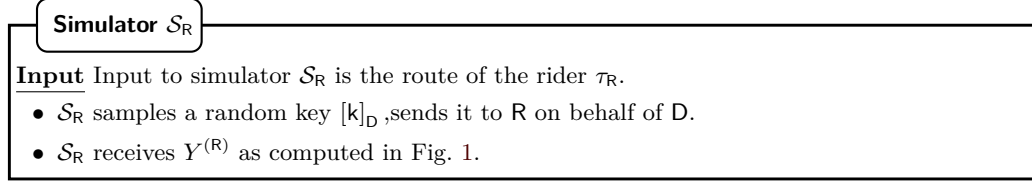
Simulator $\mathcal{S}_{SP}$

Input Input to the simulator $\mathcal{S}_{SP}$ is $\mathbf{b} = 1$ if $|\tau_R \cap \tau_D| \geq \max\{t_R, t_D\}$, 0 otherwise.

- $\mathcal{S}_{SP}$ samples a random key k , integers m, n, d such that $m, n > d$ and $d \geq \max\{t_R, t_D\}$ if $\mathbf{b} = 1$, otherwise $d < \max\{t_R, t_D\}$. SP also samples two indices $i_R \in [m - d]$ and $i_D \in [n - d]$.
- $\mathcal{S}_{SP}$ picks random routes for R and D in the following way:
 $\tau'_R = \{(v_1^{(R)}, v_2^{(R)}), \dots, (v_{i_R-1}^{(R)}, v_{i_R}^{(R)}), (v_{i_R}, v_{i_R+1}), \dots, (v_{i_R+d-1}, v_{i_R+d}^{(R)}), \dots, (v_{m-1}^{(R)}, v_m^{(R)})\}$
 $\tau'_D = \{(v_1^{(D)}, v_2^{(D)}), \dots, (v_{i_D-1}^{(D)}, v_{i_D}^{(D)}), (v_{i_D}, v_{i_D+1}), \dots, (v_{i_D+d-1}, v_{i_D+d}^{(D)}), \dots, (v_{n-1}^{(D)}, v_n^{(D)})\}$
 where, $\forall j \in [0, d - 1]$, $v_{i_R+j} = v_{i_D+j}$.
- $\mathcal{S}_{SP}$ computes
 $y_{j_R} = F(k, v_{j_R} || v_{j_R+1})$ for all $j_R \in \{i_R, \dots, i_R + d - 1\}$,
 $y_{j_D} = F(k, v_{j_D} || v_{j_D+1})$ for all $j_D \in \{i_D, \dots, i_D + d - 1\}$,
 $y_j^{(R)} = F(k, v_j^{(R)} || v_{j+1}^{(R)})$ for all $j \in \{1, \dots, i_R - 1\} \cup \{i_R + d, \dots, m - 1\}$
 $y_j^{(D)} = F(k, v_j^{(D)} || v_{j+1}^{(D)})$ for all $j \in \{1, \dots, i_D - 1\} \cup \{i_D + d, \dots, n - 1\}$
- $\mathcal{S}_{SP}$ sends
 $Y^{(R)} = \{y_1^{(R)}, \dots, y_{i_R-1}^{(R)}, y_{i_R}, \dots, y_{i_R+d-1}^{(R)}, y_{i_R+d}, \dots, y_{m-1}^{(R)}\}$ and
 $Y^{(D)} = \{y_1^{(D)}, \dots, y_{i_D-1}^{(D)}, y_{i_D}, \dots, y_{i_D+d-1}^{(D)}, y_{i_D+d}, \dots, y_{n-1}^{(D)}\}$ to SP, on behalf of R and D respectively.

Figure 3: Simulator $\mathcal{S}_{SP}$ for Π_{Int} for a corrupt SP

Indistinguishability of $\text{VIEW}_{\text{REAL}}^{\text{SP}}$ and $\text{VIEW}_{\text{IDEAL}}^{\text{SP}}$. Fig. 3 describes the simulation steps when SP is semi-honest. The simulator $\mathcal{S}_{SP}$ has the input $\mathbf{b}$ where $\mathbf{b} = 1$ if $|\tau_R \cap \tau_D| \geq \max\{t_R, t_D\}$, 0 otherwise. $\mathcal{S}_{SP}$ picks random routes τ'_R and τ'_D for R and D respectively such that $|\tau'_R \cap \tau'_D| = d$, where $d \geq \max\{t_R, t_D\}$ if $\mathbf{b} = 1$, otherwise $d < \max\{t_R, t_D\}$. It locally samples a random key k on behalf of R and D. Then, using τ'_R and τ'_D , it follows the protocol steps on behalf of R and D respectively. Consider $\text{VIEW}_{\text{REAL}}$ be the view of SP in the protocol and $\text{VIEW}_{\text{IDEAL}}$ be the simulated view of SP generated by $\mathcal{S}_{SP}$. Note that $\text{VIEW}_{\text{REAL}} = Y_R || Y_D$ and $\text{VIEW}_{\text{IDEAL}} = Y^{(R)} || Y^{(D)}$. Since, $Y_R \equiv Y^{(R)}$ and $Y_D \equiv Y^{(D)}$, as both are evaluations of PRF's. Therefore, $\text{VIEW}_{\text{REAL}} \equiv \text{VIEW}_{\text{IDEAL}}$. Hence, the protocol Π_{Int} is secure against a semi-honest SP.

Figure 4: Simulator $\mathcal{S}_R$ for Π_{Int} for a corrupt R

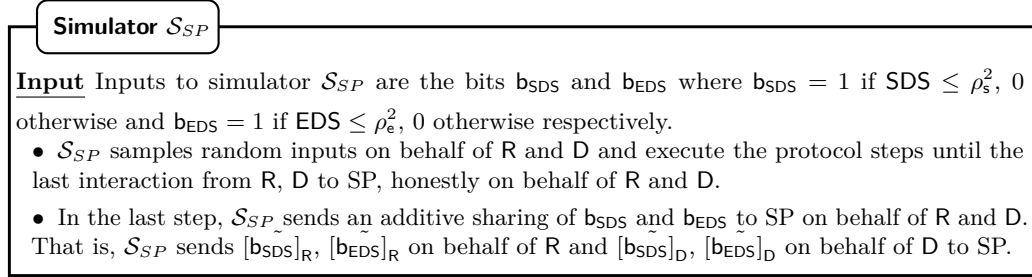
Indistinguishability of $\text{VIEW}_{\text{REAL}}^R$ and $\text{VIEW}_{\text{IDEAL}}^R$. Let $\mathcal{S}_R$ be the simulator corresponding to a corrupt rider R. The steps of $\mathcal{S}_R$ are described in Fig. 4. It interacts with R on behalf of SP and D. During the simulation, it samples a random key $[k]_D$ and sends it to R, on behalf of D. Then, on behalf of SP, it receives $Y^{(R)}$ as per protocol specification. Since D and SP do not send any messages to R, the simulated view is trivially indistinguishable.

Observe that the simulation for the corrupt driver is the same as that of the corrupt rider. Thus, we omit $\mathcal{S}_D$.

4.2 Security Proof of Π_{EndPoint}

Here, we will discuss the security proof for the end-point matching protocol. Similar to Section 4.1, we will first discuss the simulator, $\mathcal{S}_{SP}$, for corrupt SP followed by the simulator, $\mathcal{S}_R$, for corrupt R.

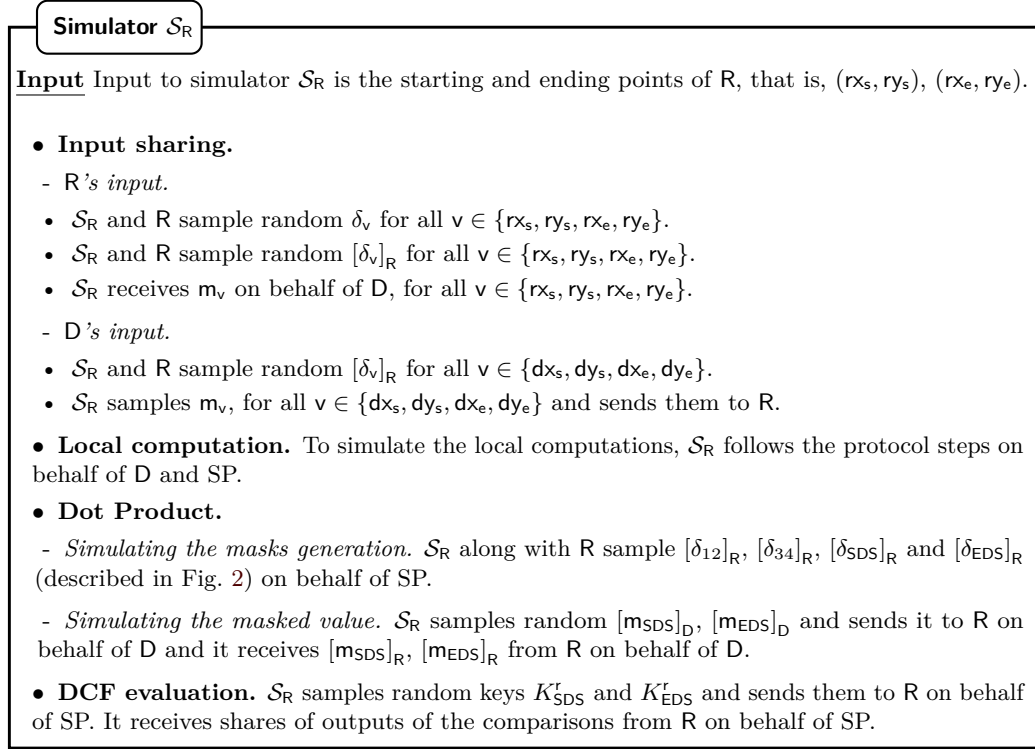
$\mathcal{S}_{SP}$ has inputs b_{SDS} and b_{EDS} where $b_{\text{SDS}} = 1$ if $\text{SDS} \leq \rho_s^2$, 0 otherwise and $b_{\text{EDS}} = 1$ if $\text{EDS} \leq \rho_e^2$, 0 otherwise.

Figure 5: Simulator $\mathcal{S}_{SP}$ for Π_{EndPoint} for a corrupt SP

Indistinguishability of $\text{VIEW}_{\text{REAL}}^{\text{SP}}$ and $\text{VIEW}_{\text{IDEAL}}^{\text{SP}}$. The simulation steps for the corrupt SP are given in Fig. 5. To simulate the view of SP, $\mathcal{S}_{SP}$ samples random inputs of R and D. With the random inputs, $\mathcal{S}_{SP}$, execute the steps of Π_{EndPoint} honestly on behalf of R and D. In the last step of the protocol, where R and D send their shares of the outputs, $\mathcal{S}_{SP}$ sends random additive sharing of b_{SDS} and b_{EDS} to SP. Note that $\text{VIEW}_{\text{REAL}}$ consists of the following.

$$\begin{aligned}
 &\text{VIEW}_{\text{REAL}} \\
 &= \{\delta_{rx_s}, \delta_{ry_s}, \delta_{rx_e}, \delta_{ry_e}, \delta_{dx_s}, \delta_{dy_s}, \delta_{dx_e}, \delta_{dy_e}, \delta_{12}, \delta_{34}, \delta_{\text{SDS}}, \delta_{\text{EDS}}, \\
 &K_{\text{SDS}}^r, K_{\text{EDS}}^r, K_{\text{SDS}}^d, K_{\text{EDS}}^d, [b_{\text{SDS}}]_R, [b_{\text{EDS}}]_R, [b_{\text{SDS}}]_D, [b_{\text{EDS}}]_D\} \\
 &\text{VIEW}_{\text{IDEAL}} \\
 &= \{\tilde{\delta}_{rx_s}, \tilde{\delta}_{ry_s}, \tilde{\delta}_{rx_e}, \tilde{\delta}_{ry_e}, \tilde{\delta}_{dx_s}, \tilde{\delta}_{dy_s}, \tilde{\delta}_{dx_e}, \tilde{\delta}_{dy_e}, \tilde{\delta}_{12}, \tilde{\delta}_{34}, \tilde{\delta}_{\text{SDS}}, \tilde{\delta}_{\text{EDS}}, \\
 &\tilde{K}_{\text{SDS}}^r, \tilde{K}_{\text{EDS}}^r, \tilde{K}_{\text{SDS}}^d, \tilde{K}_{\text{EDS}}^d, [\tilde{b}_{\text{SDS}}]_R, [\tilde{b}_{\text{EDS}}]_R, [\tilde{b}_{\text{SDS}}]_D, [\tilde{b}_{\text{EDS}}]_D\}
 \end{aligned}$$

$\mathcal{S}_{SP}$ generates the random masks and the DCF keys by honestly following the protocol; therefore, they have identical distribution. Finally, both in the simulated view as well as in the real view, SP receives additive sharing of the same values, b_{SDS} and b_{EDS} . Hence, $\text{VIEW}_{\text{REAL}} \equiv \text{VIEW}_{\text{IDEAL}}$.

Figure 6: Simulator $\mathcal{S}_R$ for Π_{EndPoint} for a corrupt R

Indistinguishability of $\text{VIEW}_{\text{REAL}}^R$ and $\text{VIEW}_{\text{IDEAL}}^R$. $\mathcal{S}_R$ has the inputs of the corrupt rider R, that is, $(rx_s, ry_s), (rx_e, ry_e)$. $\mathcal{S}_R$ needs to simulate the view of R. The simulation steps of $\mathcal{S}_R$ are provided in Fig. 6. The view of R comprises of the exchanged messages during the input sharing, dot product, and DCF evaluations. In the input sharing phase, corresponding to R's input $\mathcal{S}_R$ (on behalf of SP) and R, sample the masks for the inputs and the shares of the masks. Then R sends the masked values to D, $\mathcal{S}_R$ receives the masked values on behalf of D. Corresponding to D's inputs, on behalf of SP, $\mathcal{S}_R$ and R sample shares of the random masks. Here note that D's inputs are not known to $\mathcal{S}_R$. However, it needs to prepare the masked values on behalf of D and send them to R. $\mathcal{S}_R$ samples random values as the masked values and sends them to D. Since the masks are unknown to R, the masked values are indistinguishable from randomly sampled values. Thus, this step of the simulation is indistinguishable. To perform the dot product, SP generates correlated randomness $\delta_{12}, \delta_{34}, \delta_{SDS}$ and δ_{EDS} . These values are additively shared between R and D. Therefore, on behalf of SP, $\mathcal{S}_R$ along with R sample random values $[\delta_{12}]_R, [\delta_{34}]_R, [\delta_{SDS}]_R$ and $[\delta_{EDS}]_R$. To complete the computation of the dot product, R and D need the masked values, m_{SDS} and m_{EDS} . $\mathcal{S}_{SP}$ sends uniformly sampled $[m_{SDS}]_D$ and $[m_{EDS}]_D$ to R and it receives $[m_{SDS}]_R$ and $[m_{EDS}]_R$ on behalf of D. All these messages in the protocol are also uniformly distributed; hence, the simulated view is indistinguishable from the view of R. Finally, for the DCF evaluations, the keys K_{SDS}^r and K_{EDS}^r are uniformly distributed. Therefore, $\mathcal{S}_R$ sends uniformly sampled keys, K_{SDS}^r and K_{EDS}^r to R on behalf of SP and it receives the shares of the outputs of the comparisons from R. The simulated view is indistinguishable from the view of R, therefore, $\text{VIEW}_{\text{REAL}} \equiv \text{VIEW}_{\text{IDEAL}}$. The simulator for the corrupt driver, $\mathcal{S}_D$, is similar to $\mathcal{S}_R$.

5 Matching Multiple Pairs

In Section 3.1 and Section 3.2, we describe two different procedures to check if a rider and a driver are a match. Let $\Pi(R, D)$ indicate if R and D are a match or not, where Π can be instantiated using Π_{Int} or Π_{EndPoint} . SP obtains the output, $\Pi(R, D)$ for every pair R and D . Note that it is possible that some riders may be matched with a common driver, where one of the riders may have a match with another driver, but the other rider does not have any other matches. In that case, random assignments of rider-driver pairs may lead to starvation of some users (remain unmatched). We elaborate on this with an example below.

Consider the following scenario with 2 riders, R_1, R_2 and 2 drivers, D_1, D_2 such that $\Pi(R_1, D_1) = 1$, $\Pi(R_1, D_2) = 1$, $\Pi(R_2, D_1) = 1$ and $\Pi(R_2, D_2) = 0$. If R_1 and D_1 are matched, then both R_2 and D_2 will remain unmatched. However, (R_1, D_2) and (R_2, D_1) is an optimal match. Given a set of riders and drivers, it is possible to find the optimal matching by executing a maximum bipartite matching. Since SP gets the output of the matching protocol for every pair in clear, it can construct the adjacency matrix of the corresponding bipartite graph and execute the matching algorithm locally. Therefore, any state-of-the-art maximum matching algorithm can be used to obtain the optimum matching without incurring any communication overhead. Furthermore, the computation overhead will be towards SP only, depending on the underlying matching algorithm. Also, in the case of the intersection based matching algorithm, it is easy to obtain a preference list (based on the cardinality of the intersection set) corresponding to every rider and driver. For a rider R_i , let $\mathcal{D}^{(i)} = \{D_1^{(i)}, \dots, D_{m_i}^{(i)}\}$ be the preference list. That is, $\forall j \in [m_i]$, $f(\tau_{R_i}, t_{R_i}, \tau_{D_j}, t_{D_j}) = 1$ and $|\tau_{R_i} \cap \tau_{D_j}| \geq |\tau_{R_i} \cap \tau_{D_{j+1}}|$ for all $j \in [m_i - 1]$. Similarly, driver D_j has the preference list $\mathcal{R}^{(j)} = \{R_1^{(j)}, \dots, R_{n_j}^{(j)}\}$. Then, SP can also locally execute the stable matching algorithm [GS62].

Protocol $\Pi_{\text{Mult-Match}}(\mathcal{R}, \mathcal{D}, \text{SP})$

Parameters:

- $\mathcal{R}$ = Set of riders and $\mathcal{D}$ = Set of drivers.

Protocol:

Key setup: For every $R \in \mathcal{R}$ and $D \in \mathcal{D}$ establish a secure communication channel and perform key setup by executing $\Pi_{\text{KeySetup}}(\mathcal{R}, \mathcal{D})$. For pair $(R, D) \in \mathcal{R} \times \mathcal{D}$, k_{RD} is common key.

Matching: For every $R \in \mathcal{R}$ and $D \in \mathcal{D}$, perform $\Pi_{\text{Match}}(R, D, SP)$ in parallel. SP stores the output at $M[R, D]$.

Local matching by SP:

- SP constructs a bipartite graph $G = (V, E)$ where $V = \mathcal{R} \cup \mathcal{D}$ and $(R, D) \in E$ if and only if $M[R, D] = 1$.
- SP performs a maximum matching on G and let $G' = (V, E')$ be the maximum matching subgraph of G .
- If $(R, D) \in E'$, then SP sends (match, D) to R and (match, R) to D .

Output:

- If $R \in \mathcal{R}$ is matched then it outputs (match, D) for some $D \in \mathcal{D}$, else it outputs **unmatched**.
- If $D \in \mathcal{D}$ is matched then it outputs (match, R) for some $R \in \mathcal{R}$, else it outputs **unmatched**.
- Let $\mathcal{R}'$ and $\mathcal{D}'$ are the set of unmatched riders and drivers. SP outputs $\mathcal{R}', \mathcal{D}'$.

Figure 7: Multiple ride-sharing matching

When a rider (driver) raises a ride (match) request, it is checked with all the available drivers (riders) to see if they are a match. We consider a bipartite graph, $G = (V, E)$ where $V = \mathcal{R} \cup \mathcal{D}$, $\mathcal{R}$ and $\mathcal{D}$ are the set of all riders and drivers, respectively. Note that,

$\mathcal{R}$ and $\mathcal{D}$ form a partition of V . We consider $(R, D) \in E$ to be an edge in the graph if $R \in \mathcal{R}$ and $D \in \mathcal{D}$ are matched, that is, $\Pi_{\text{Match}}(R, D) = 1$. Thus, we execute a maximum bipartite matching algorithm on G to obtain the maximum number of matches. It is easy to see that a maximum matching in G will give a maximum possible matching between the set of riders and drivers. We represent a single rider-driver matching by Π_{Match} , this can be realized by both Π_{Int} as well as Π_{EndPoint} . We provide the complete construction ($\Pi_{\text{Mult-Match}}$) of multiple rider-driver matching in Fig. 7. Note that here, we consider the number of riders and the number of drivers to be the same, although it can be generalized to an arbitrary number of riders and drivers. A set of riders and drivers, along with the service provider SP, execute $\Pi_{\text{Mult-Match}}$.

6 Experiments and Results

In this section, we evaluate the effectiveness of QuickPool in providing an efficient ride-sharing solution. Firstly, we describe the implementation details of QuickPool followed by benchmark results of our proposed intersection based matching and end-point based matching protocols. We show that, compared to previous works [PGT⁺19], our protocols achieve significant speed-up without compromising user privacy.

6.1 Implementation Details

We now outline the implementation details of QuickPool, which we utilize for benchmarking the intersection based matching and end-point based matching protocols. For benchmarking we use synthetic data in our experiments to obtain the communication and runtime costs. We would like to emphasize that using random inputs is equivalent to using real inputs, as all operations are performed obliviously and the underlying data distribution does not impact the computation or communication costs.

Environment. The benchmarks are conducted over MAN network simulated on a local server, equipped with a 4.5GHz, 24 cores, 48 threads, AMD Ryzen™ Threadripper PRO 5965WX processor having 256GB RAM. Each rider and driver as well as the SP are executed as multiple processes on a single server with multithreading enabled. More details are provided in Appendix C.2. We consider a bandwidth of 500 Mbps and 10 ms latency and use the Linux `tc` command from network emulation package `netem` to emulate a MAN network for inter-process communication. Both intersection based matching and end-point based matching protocols are benchmarked in MAN, as QuickPool, being a ride-sharing solution, is typically more suited for MAN environments. Each test suite is executed ten times without interruption and the average run time is reported in this paper.

Implementation of QuickPool. We implement QuickPool¹ in C++17, leveraging the codebase of Asterisk [KKP⁺24] and EMP toolkit [WMK16]. We set the computational security parameter $\kappa = 128$ and ensure a statistical security of at least 2^{-40} . To implement the intersection based matching protocol, we utilize the AES implementation from the EMP toolkit, which internally utilizes highly efficient AES-NI. We utilize `funshade` [ICÖ22] implementation for secure comparisons used in end-point based matching.

Implementation for comparison with other works. We compare our intersection based matching protocol with the O-PrivatePool from TOPPool [PGT⁺19], which used the BaRK-OPRF PSI protocol [KKRT16]. The BaRK-OPRF PSI protocol is recognized as one of the fastest state-of-the-art PSI protocols for large batches of data. Although O-PrivatePool codebase is publicly available [Gun18a] [Gun18b], we are unable to run it due to compilation issues. Hence, we base our comparison on the BaRK-OPRF PSI protocol (utilizing the open-source code accessible at [Tri16]). Given that O-PrivatePool

¹Our implementation is publicly available at: <https://github.com/cris-coders-iisc/QuickPool>. Our code is developed for benchmarking and is not optimized for industry-grade use.

relies on BaRK-OPRF, we assert that O-PrivatePool will take at least as much time as BaRK-OPRF. We compare our end-point based matching protocol with that of TOPPool, which relies on AHE (Paillier’s encryption scheme [Pai99]). As aforementioned, since their codebase has compilation issues, we implement a similar matching algorithm using a C++-based Paillier encryption library available online [Saf22], to compare with our work.

6.2 Performance Benchmarks of QuickPool

We now benchmark the intersection-based and end-point-based matching protocols, highlighting their efficiency in terms of both communication cost and runtime.

Intersection based matching. We analyze the performance of the intersection based matching protocol (Section 3.1, Fig. 1) by setting the number of riders equal to the number of drivers and varying both from 1 to 100, as depicted in Fig. 8. Additionally, we also vary the set size from 32 up to 4096, where by ‘set size’ we mean the route length of each user. We observe that even with a set size of up to 2048, considering the number of riders and drivers to be up to 100, per party communication cost remains almost within 3MB. Note that in this protocol, SP does not send any data to any other party throughout the process. Regarding runtime, for all considered set sizes and varying numbers of riders and drivers, the total runtime remains within 35 seconds (roughly half a minute).

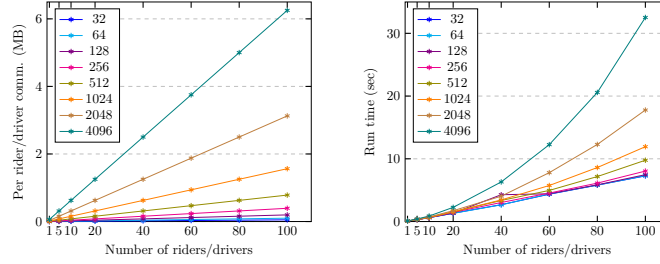


Figure 8: Performance of intersection based matching algorithm over a MAN network, for varying numbers of riders/drivers in $\{1, 5, 10, 20, 40, 60, 80, 100\}$ and set size in $\{2^j\}_{j \in [5, 12]}$.

Additionally, we report the throughput for a set size of 256 in Table 3, which captures the number of rider-driver pairs matched per minute and is computed as throughput (/minute) = $\min(\# \text{ riders}, \# \text{ drivers}) * 60 / \text{run time}$.

End-point based matching. Similar to the intersection based matching protocol, we analyze the performance of the end-point based matching protocol (Section 3.2, Fig. 2) by varying the number of riders and drivers, as depicted in Fig. 9.

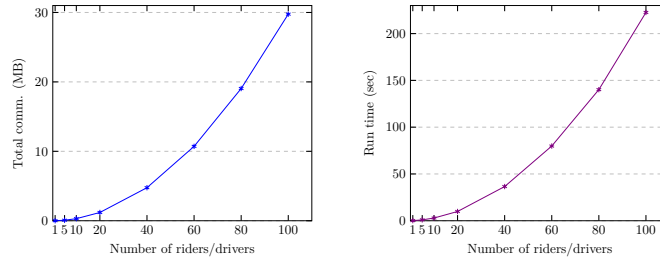


Figure 9: Performance of end-point based matching algorithm over a MAN network, for varying numbers of riders/drivers in $\{1, 5, 10, 20, 40, 60, 80, 100\}$.

We observed that even with 100 riders and drivers, the total communication remains below 30MB, while with up to 40 riders and drivers, the communication cost is less than 5MB. Notably, in this protocol, each rider and driver transmits significantly very less data compared to SP. For instance, in a scenario with 100 riders and drivers, each of the riders

and drivers needs to send only $\sim 6.25\text{KB}$, as shown in Table 2. Note that the amount of data sent by SP is comparatively higher, as it needs to transmit the DCF keys to both the rider and the driver.

Table 2: Communication cost (in KB) of SP, riders and drivers in end-point based matching over MAN for varying numbers of users (riders/drivers).

# riders/ drivers	1	5	10	20	40	60	80	100
sent by SP	2.95	73.16	292.42	1169.22	4675.94	10520.16	18701.88	29221.09
sent by each user	0.06	0.31	0.63	1.25	2.50	3.75	5.00	6.25

Regarding runtime, this protocol completes its execution very quickly. For instance, with up to 40 riders and drivers, it takes less than 1 minute to complete. Additionally, we report the throughput in Table 3 for varying number of riders and drivers. Note that although the throughput decreases with the increase of users due to increased computation and runtime, the likelihood of successful matching in real scenarios increases. This is because there are more matching options available for each rider/driver.

Table 3: Run time and throughput (per minute) for intersection (considering set size = 256) and end-point based matching algorithms for varying numbers of riders/drivers.

# riders/ drivers	Intersection based		End-point based	
	Run time (sec)	Throughput (/min)	Run time (sec)	Throughput (/min)
5	0.31	967.74	0.94	319.15
10	0.64	937.50	2.89	207.61
20	1.46	821.92	9.89	121.33
40	3.04	789.47	36.54	65.68
60	4.54	792.95	79.83	45.10
80	6.12	784.31	140.11	34.26
100	8.04	746.27	222.60	26.95

6.3 Comparison with Previous Works

The BaRK-OPRF protocol operates between a single pair of parties for computing private set intersection. We conducted experiments with BaRK-OPRF, varying the set size from 32 to 4096, and compared both of its per party communication cost and runtime with our intersection-based matching protocol for a single rider and driver. Our observations indicate that our protocol significantly outperforms BaRK-OPRF. For instance, with a set size of 4096, our protocol’s execution time is $\sim 100\text{ms}$, while even for a set size of 32, BaRK-OPRF takes with $\sim 140\text{ms}$. Fig. 10 illustrates this comparison for different set sizes. We observe that QuickPool achieves $1.7 - 2.2\times$ and at least $8\times$ improvement over BaRK-OPRF in terms of run time and communication cost respectively.

For end-point based matching, we compare the performance of our protocol with TOPPool for a single rider-driver by varying ρ in $\{4, 8, 16\}$, where ρ represents the maximum threshold for start and end distances. While the performance of TOPPool’s end-point based matching depends on ρ , our protocol maintains consistent performance regardless of its value. As shown in Fig. 3, even for the smallest value of ρ , our protocol achieves a $4.5\times$ reduction in communication cost and a $1.10\times$ improvement in runtime compared to TOPPool. Moreover, as ρ increases, the improvement ratio grows quadratically for both communication cost and runtime. Although TOPPool addresses only the single rider-driver matching case, we extend our analysis to the multi rider-driver scenario, with details provided in Appendix C.1.

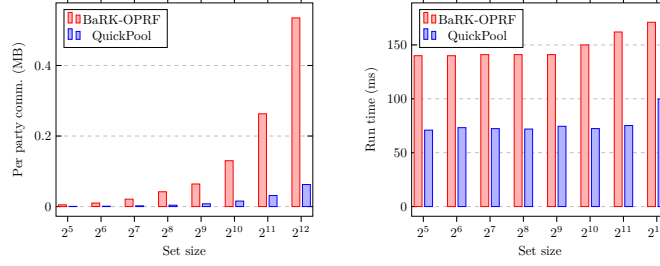


Figure 10: Performance comparison over a MAN network between BaRK-OPRF and QuickPool for a single rider and driver, for varying set size in $\{2^j\}_{j \in [5,12]}$.

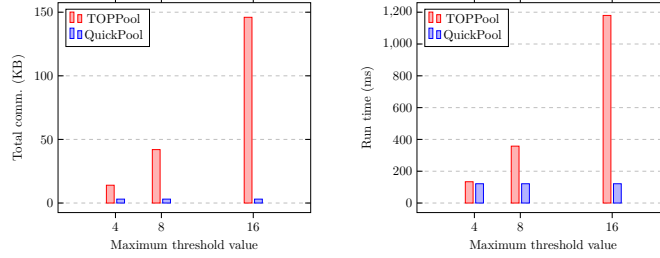


Figure 11: Performance comparison over a MAN network between the end-point based matching protocols of TOPPool and QuickPool for a single rider and driver, for varying maximum threshold value in $\{4, 8, 16\}$.

7 Conclusion and Future Work

In this work, we construct privacy-preserving protocols for both intersection based and end-point based driver-rider matching that are more efficient compared to prior works. We also demonstrate their effectiveness for multiple simultaneous matching. To the best of our knowledge, this is the first work that answers how to *simultaneously match* multiple ride requests in a privacy-preserving manner. Our protocols can be integrated into a provider platform to build secure applications. We list a few steps needed to make the protocols into fully usable practical solutions.

Choice of protocol. End-point and intersection based matching protocols have distinct use cases and are meant to complement each other. The choice of which algorithm to use can be left to the users, allowing them to select the most suitable algorithm in their apps based on their needs.

Time matching. TOPPool [PGT⁺19] proposes matching a rider and a driver, not just with either the intersection or end-point matching, but also their respective ride times. To facilitate the time-based matching in QuickPool, we need to perform one additional DCF to compare the differences between the rider’s and the driver’s ride times with a publicly known end-user-specific threshold value. The result of the comparison can be used as a match criteria together with either intersection based or end-point based matching.

Group matching. In group matching [YZY⁺21], multiple riders who satisfy the matching criteria (significant overlap in the respective routes or nearby start and end-points) are considered as a single group. Then, a representative of a group is matched with a driver. One can trivially extend the rider-driver matching protocol in QuickPool to perform group matching. First, we compare pairs of riders to obtain a matched group. Next, a representative of this matched group can execute a matching protocol with the drivers. Here, the matching protocol may not necessarily be ride-sharing matching. One can use ride-hailing matching as well where the driver’s location should be close to the rider’s locations. For this, we can use a relaxed version of end-point based matching where the distance between only the start locations of rider and driver is less than a threshold value.

Other desirable features. A secure and anonymous payment system such as eCash or equivalent needs to be integrated into the system for fare payment after ride completion. Similar to features available in existing ride matching services, an anonymous rating system to mutually rate riders and drivers can be implemented for quality improvement and reputation maintenance.

8 Bibliography

References

- [AHHK18] Ulrich Matchi Aïvodji, Kévin Huguenin, Marie-José Huguet, and Marc-Olivier Killijian. Sride: A privacy-preserving ridesharing system. In *Proceedings of the 11th ACM Conference on Security & Privacy in Wireless and Mobile Networks*, 2018.
- [BCG⁺21] Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Nishant Kumar, and Mayank Rathee. Function secret sharing for mixed-mode and fixed-point secure computation. In *Advances in Cryptology – EUROCRYPT 2021*, 2021.
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015.
- [BLM⁺19] Mohamed Baza, Nouredine Lasla, Mohamed MEA Mahmoud, Gautam Srivastava, and Mohamed Abdallah. B-ride: Ride sharing with privacy-preservation, trust and fair payment atop public blockchain. *IEEE Transactions on Network Science and Engineering*, 2019.
- [BOGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, 1988.
- [CCD88] David Chaum, Claude Crépeau, and Ivan Damgard. Multiparty unconditionally secure protocols. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, 1988.
- [CDH10] Octavian Catrina and Sebastiaan De Hoogh. Improved primitives for secure multiparty integer computation. In *Security and Cryptography for Networks*, 2010.
- [GS62] David Gale and Lloyd S Shapley. College admissions and the stability of marriage. *The American mathematical monthly*, 1962.
- [Gun18a] Gunnar Gunnarsson. Bark-oprf protocol modified for a ridesharing context, 2018. URL: <https://github.com/GunnarGunnarsson/bark-rideshare>.
- [Gun18b] Gunnar Gunnarsson. O-privatepool: An optimized version of privatepool, 2018. URL: <https://github.com/GunnarGunnarsson/oprivatepool>.
- [HKK⁺22] Aditya Hegde, Nishat Koti, Varsha Bhat Kukkala, Shravani Patil, Arpita Patra, and Protik Paul. Attaining god beyond honest majority with friends and foes. In *International Conference on the Theory and Application of Cryptology and Information Security*, 2022.

- [HLF⁺21] Junxin Huang, Yuchuan Luo, Shaojing Fu, Ming Xu, and Bowen Hu. pride: Privacy-preserving online ride hailing matching system with prediction. *IEEE Transactions on Vehicular Technology*, 2021.
- [HLX⁺22] Junxin Huang, Yuchuan Luo, Ming Xu, Bowen Hu, and Jian Long. pshare: Privacy-preserving ride-sharing system with minimum-detouring route. *Applied Sciences*, 2022.
- [HOS17] Per Hallgren, Claudio Orlandi, and Andrei Sabelfeld. Privatepool: Privacy-preserving ridesharing. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017.
- [ICÖ22] Alberto Ibarrondo, Hervé Chabanne, and Melek Önen. Funshade: Functional secret sharing for two-party secure thresholded distance evaluation. *Cryptology ePrint Archive*, 2022.
- [KKP⁺24] Banashri Karmakar, Nishat Koti, Arpita Patra, Sikhar Patranabis, Protik Paul, and Divya Ravi. Asterisk: Super-fast mpc with a friend. In *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.
- [KKPRG22] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, and Bhavish Raj Gopal. Pentagod: Stepping beyond traditional god with five parties. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.
- [KKRT16] Vladimir Kolesnikov, Ranjit Kumaresan, Mike Rosulek, and Ni Trieu. Efficient batched oblivious prf with applications to private set intersection. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016.
- [KMV22] Deepak Kumaraswamy, Shyam Murthy, and Srinivas Vivek. Revisiting driver anonymity in oride. In Riham AlTawy and Andreas Hülsing, editors, *Selected Areas in Cryptography*, pages 25–46, Cham, 2022. Springer International Publishing.
- [LWD23] Qingyuan Li, Hao Wu, and Chen Dong. A privacy-preserving ride matching scheme for ride sharing services in a hot spot area. *Electronics*, 2023.
- [LZL19] Meng Li, Liehuang Zhu, and Xiaodong Lin. Efficient and privacy-preserving carpooling using blockchain-assisted vehicular fog computing. *IEEE Internet of Things Journal*, 6(3):4573–4584, 2019. doi:10.1109/JIOT.2018.2868076.
- [MAL23] Daniel Morales, Isaac Agudo, and Javier Lopez. Private set intersection: A systematic literature review. *Computer Science Review*, 2023.
- [McK22] McKinsey and Company. <https://www.mckinsey.com/features/mckinsey-center-for-future-mobility/mckinsey-on-urban-mobility/snap-shot-of-the-european-car-sharing-market>, 2022.
- [MR22] Francesca Martelli and Maria Elena Renda. Enhancing privacy in ride-sharing applications through pois selection. In *2022 IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2022.
- [MSD⁺24] Tapaswini Mohanty, Vikas Srivastava, Sumit Kumar Debnath, Ashok Kumar Das, and Biplab Sikdar. Quantum secure threshold private set intersection protocol for iot-enabled privacy-preserving ride-sharing application. *IEEE Internet of Things Journal*, 11(1):1761–1772, 2024. doi:10.1109/JIOT.2023.3291132.

- [MV22] Shyam Murthy and Srinivas Vivek. Passive triangulation attack on oride. In Alastair R. Beresford, Arpita Patra, and Emanuele Bellini, editors, *Cryptology and Network Security*, pages 167–187, Cham, 2022. Springer International Publishing.
- [Nor20] NortonLifeLock. Uber Announces New Data Breach Affecting 57 million Riders and Drivers. <http://lifelock.norton.com/learn/data-breaches/uber-data-breach-affects-57-million-rider-and-driver-accounts>, 2020.
- [Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*, 1999.
- [PGT⁺19] Elena Pagnin, Gunnar Gunnarsson, Pedram Talebi, Claudio Orlandi, and Andrei Sabelfeld. Toppool: Time-aware optimized privacy-preserving ridesharing. *Proceedings on Privacy Enhancing Technologies*, 2019.
- [PMLC25] David Palma, Pier Luca Montessoro, Mirko Loghi, and Daniele Casagrande. A privacy-preserving system for confidential carpooling services using homomorphic encryption. *Advanced Intelligent Systems*, 7(5):2400507, 2025. URL: <https://advanced.onlinelibrary.wiley.com/doi/abs/10.1002/aisy.202400507>, doi:10.1002/aisy.202400507.
- [PSSY21] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. ABY2.0: Improved mixed-protocol secure two-party computation. In *30th USENIX Security Symposium (USENIX Security 21)*, 2021.
- [Saf22] Safeheron. crypto-paillier-cpp: Paillier cryptosystem library in c++, 2022. URL: <https://github.com/Safeheron/crypto-paillier-cpp>.
- [SASG15] Mitja Stiglic, Niels Agatz, Martin Savelsbergh, and Mirko Gradisar. The benefits of meeting points in ride-sharing systems. *Transportation Research Part B: Methodological*, 82:36–53, 2015. URL: <https://www.sciencedirect.com/science/article/pii/S0191261515002088>, doi:10.1016/j.trb.2015.07.025.
- [SMDF16] David Sánchez, Sergio Martínez, and Josep Domingo-Ferrer. Co-utile p2p ridesharing via decentralization and reputation management. *Transportation Research Part C: Emerging Technologies*, 2016.
- [SRML17] Ahmed B. T. Sherif, Khaled Rabieh, Mohamed M. E. A. Mahmoud, and Xiaohui Liang. Privacy-preserving ride sharing scheme for autonomous vehicles in big data era. *IEEE Internet of Things Journal*, 4(2):611–618, 2017. doi:10.1109/JIOT.2016.2569090.
- [Tec23] TechCabal. My driver the stalker. <https://techcabal.com/2023/09/15/my-driver-the-stalker/>, 2023.
- [The18] The Wall Street Journal. <https://www.wsj.com/articles/uber-to-pay-148-million-penalty-to-settle-2016-data-breach-1537983127>, 2018.
- [the19] thejournal.ie. West Dublin gang using hailing apps to target older taxi drivers. <https://www.thejournal.ie/west-dublin-taxi-robbery-4420178-Jan-2019/>, 2019.

- [Tri16] Ni Trieu. Github - osu-crypto/BaRK-OPRF: Efficient Batched Oblivious PRF with Applications to Private Set Intersection (CCS 2016). <https://github.com/osu-crypto/BaRK-OPRF>, 2016.
- [WMK16] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [WZL⁺18] Fengwei Wang, Hui Zhu, Ximeng Liu, Rongxing Lu, Fenghua Li, Hui Li, and Songnian Zhang. Efficient and privacy-preserving dynamic spatial query scheme for ride-hailing services. *IEEE Transactions on Vehicular Technology*, 2018.
- [XZZ⁺22] Qi Xu, Hui Zhu, Yandong Zheng, Jiaqi Zhao, Rongxing Lu, and Hui Li. An efficient and privacy-preserving route matching scheme for carpooling services. *IEEE Internet of Things Journal*, 2022.
- [YJZ⁺21] Haining Yu, Xiaohua Jia, Hongli Zhang, Xiangzhan Yu, and Jiangang Shu. Psride: Privacy-preserving shared ride matching for online ride hailing systems. *IEEE Transactions on Dependable and Secure Computing*, 2021.
- [YZY⁺21] Haining Yu, Hongli Zhang, Xiangzhan Yu, Xiaojiang Du, and Mohsen Guizani. Pgride: Privacy-preserving group ridesharing matching in online ride hailing services. *IEEE Internet of Things Journal*, 2021.
- [ZHQ25] En Zhang, Yuchen Huang, and Leiyong Qin. Privacy-preserving matching scheme for iov-enabled ride-sharing services. *IEEE Transactions on Vehicular Technology*, 2025.
- [ZLL⁺25] Zhirun Zheng, Zhetao Li, Saiqin Long, Suiming Guo, Chao Chen, and Ke Xu. Pricing utility vs. location privacy: A differentially private data sharing framework for ride-on-demand services. *IEEE Transactions on Dependable and Secure Computing*, pages 1–17, 2025. doi:10.1109/TDSC.2025.3532599.
- [ZWH⁺23] Juyuan Zhang, Licheng Wang, Xiaoya Hu, Ruiqin Li, and Shihui Zheng. Privacy-preserving online ride-hailing service system based on taking the intersection of private sets of points of interest. In *2023 International Conference on Mobile Internet, Cloud Computing and Information Security (MICCIS)*, 2023.
- [ZZWC22] Quan Zhou, Zhikang Zeng, Kemeng Wang, and Menglong Chen. Privacy protection scheme for the internet of vehicles based on private set intersection. *Cryptography*, 2022.

A Related Works

The works in PP-RSS can be broadly classified as those that operate in an SP-aided setting and others that do not have an SP. Another classification is the set of works that use an auxiliary server to help make their protocol efficient. The third are those that use public-key primitives. Finally, we list works that leak more information to riders than just the specific matched driver which might lead to location-harvesting attacks.

A very early work by Sherif et al. [SRML17] looks at an SP-aided model using autonomous vehicles (AV) to provide PP-RSS. In their model, users are categorized as ‘primary’ and ‘secondary’ where the primary user is an all-powerful entity having full right of use of AV and secondary users share the AV with the primary user, and the SP is the adversary from whom user location information is kept hidden. Another early PP-RSS

protocol PrivatePool, but SP-less, by Hallgren et al. [HOS17] obviously matches riders based on proximity matching of end points as well as overlaps in riders' route trajectories. Their work does not take into account the start time of travel and does not rely on a central SP for ride matching. TOPPool [PGT⁺19] is a follow-up work of PrivatePool, by Pagnin et al. that incorporates the time of travel for ride matching. It used private set intersection for route segment matching and used an additive homomorphic encryption scheme along with a privacy-preserving matching method to aid in oblivious end-point matching. The works of Baza et al. [BLM⁺19] and Sanchez et al. [SMDF16] are both decentralized models, with the former using public blockchain while the latter used peer trust enforcement using a distributed reputation management protocol.

The work of Li et al. [LZL19] has the following entities : trusted authority, cloud service provider, road-side-fog-computing-units, riders, and drivers. They provide efficient matching while using a blockchain to store their records. However, the presence of three non-colluding trust entities entails stronger assumptions. The works PSRide by Yu et al. [YJZ⁺21] and pShare by Huang et al. [HLX⁺22] made use of two non-colluding servers, namely the SP Server and a Crypto Server, to securely evaluate Garbled Circuits for ride matching. The work by Li et al. [LWD23] used road network embedding (RNE) to represent the road network and made use of additive homomorphic encryption (AHE) to securely compute distances between riders, and used an auxiliary non-colluding server for decryption. PGRide by Yu et al. [YZY⁺21] matched a group of riders based on the smallest aggregate distance, aided by an external proxy server, using somewhat homomorphic encryption (SHE) to achieve matching. A recent work by Zhang et al. [ZHQ25] proposed a PP-RSS solution in a malicious adversarial setting and considered path intersection of multiple simultaneous riders. Their protocol internally uses OPRF which has computation overhead, and also makes use of a non-colluding auxiliary trusted authority responsible for system initialization and registration. A recent work by Palma et al. [PMLC25] uses the CKKS homomorphic encryption scheme to hide user locations from SP. However, they have a strong assumption of a fully trustworthy key-manager.

Zheng et al. [ZLL⁺25] examine the price effects of noise addition in differential privacy anonymization and propose a dynamic pricing aware privacy preserving ride-matching system. Martelli et al. [MR22] compared location masking techniques like k -anonymity, l -diversity, and obfuscation, and showed that l -diversity using actual locations as PoIs gives better performance. QuTPSI by Mohanty et al. [MSD⁺24] propose the first quantum cryptography based threshold-PSI and adapts it for use in PP-RSS. While the protocol is post-quantum secure, it requires quantum hardware technologies for practical realization.

SRide [AHHK18] by Aïvodji et al. made use of secure multi-party computation (MPC) together with SHE to obviously match riders and drivers, and provided results for intersection based matching. In their solution, a rider gets to know all matched drivers which can lead to potential location-harvesting attacks by colluding riders, for example, [MV22]. A recent work called TAROT by Xu et al. [XZZ⁺22] made use of Goldwasser-Micali (GM) public-key encryption scheme to match end-users using reverse minhash-based Jaccard similarity. Here again, the end-user decrypts all the encrypted matching results using her public key, which can potentially lead to adversarial attacks.

B Preliminaries

RSS matching paradigms: In RSS, two ride matching paradigms are considered in the literature [HOS17, AHHK18, PGT⁺19]:

- Intersection based matching
- End-point based matching

At a high level, the intersection based matching protocol takes as inputs the routes of the rider and the driver, and returns a successful match if the length of the contiguous common path of the two parties is greater than a well-known threshold. The term route will be explained in more detail in the following sections.

The end-point based matching protocol uses start and end points of the rider and driver to check for the proximity of the respective points to be within a well-known threshold distance to be deemed matchable. We use Euclidean distance (**ED**) as the metric. Given two points $X = (x_1, x_2)$ and $Y = (y_1, y_2)$, $\text{ED}(X, Y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$. Both schemes are described in detail in the following sections.

In the privacy-preserving versions of the protocols, the encryptions of the routes as well as end-points are provided as inputs to the protocols. In this paper, we provide efficient implementations for privacy-preserving versions of both protocols. We do not intend to compare the features of one protocol versus the other. Both protocols are run simultaneously, and the choice of which one of the two protocol results to use is left to the rider.

Grid based operation: For scalability and ease of operation, SP partitions its purview of operation (e.g., a city and its suburbs) into suitable size grids such that the anonymity of parties inside the grid is well-preserved, similar to other works [WZL⁺18, HLF⁺21]. SP keeps track of the grid ID of drivers when they advertise their availability for a ride (match request). Similarly, when the rider sends a ride request, SP learns the grid ID. SP forwards the ride request to drivers in the rider's grid and compares the rider information with driver information in the same grid, for matching.

Points of interest (PoI) are geographical markers [MR22, SASG15], like well-known landmarks, road intersection points, etc., each with its own unique ID. During the ride matching process, only encryptions of PoI IDs and their respective encrypted co-ordinates are shared by users. Details of each PoI are published by the SP (and available in the mobile app provided by the SP) along with its unique ID (which is represented as 64-bit integers) as well as its precise latitude and longitude coordinates. These (x, y) co-ordinates are represented in the UTM² format, where x and y are 32-bit non-negative integers, and represented in $\mathbb{Z}_{2^k}$ with a suitable value of k . For start and end locations, the app considers the PoI nearest to the users' actual location. We note here that PoIs give an additional location anonymity even after the ride is matched.

Route is an ordered set of PoIs of a rider or a driver, on the path from its start location to end location. A route is represented as a sequence of connected route segments between each PoI in the path. In other words, a route $\tau = \{(v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)\}$, where each v_i is encryption of PoI ID.

Private set intersection (PSI) is a cryptographic tool that allows two parties to compute the intersection of their respective sets without revealing any information about their individual sets to each other. [MAL23] gives a literature survey of works on PSI. PSI has been used by many schemes in ride-sharing as well as ride-hailing settings [HOS17, PGT⁺19, ZZWC22, ZWH⁺23]. Prior works [HOS17, PGT⁺19] rely on this primitive to execute the intersection based matching protocols. Here, a rider and a driver execute the role of the two parties of PSI, and their respective routes are the input sets to the PSI protocol.

Secure multi-party computation (MPC) [BOGW88, CCD88] allows multiple distrusting parties to jointly and securely compute a common function of their inputs, without revealing the inputs or intermediate outputs to any of the other parties, and revealing only the final output to all parties. In order to accomplish this, each party creates shares of their input, in a way that the original data is revealed only when a specific number of shares (also known as threshold) are put together. Next, in the secret-sharing phase, each party receives a share from each of the other parties. Finally, they jointly proceed to

²Universal Transverse Mercator coordinate system

compute the function with the help of their shares. The result is obtained by each party in secret-shared format which is reconstructed to obtain the result in plaintext format.

An MPC protocol Π_f be an n -party protocol that computes f is said to be secure, that is, it ensures correctness and privacy if for every adversary $\mathcal{A}$ there is a probabilistic polynomial time simulator $\mathcal{S}_{\mathcal{A}}$ that takes input and output of the adversary as input generates a view that is indistinguishable from the view of the adversary $\mathcal{A}$ in the real protocol. Consider $\text{VIEW}_{\text{IDEAL}}$ be that view generated by the simulator, that is, $\text{VIEW}_{\text{IDEAL}} = \{\mathcal{S}_{\mathcal{A}}(x_{\mathcal{A}}, f(x_1, x_2, \dots, x_n))\}$ where $x_{\mathcal{A}}$ is the input of $\mathcal{A}$, x_i is the input of P_i and $f(x_1, x_2, \dots, x_n)$ is the output. Let $\text{VIEW}_{\text{REAL}}$ be the adversary's view in the real protocol. Therefore, π_f securely realizes f if

$$\text{VIEW}_{\text{IDEAL}} \equiv \text{VIEW}_{\text{REAL}}$$

Function secret sharing (FSS) is a tool introduced by Boyle et al. [BGH15] that facilitates evaluating a function f in a distributed manner. FSS splits the function into pieces such that the evaluations at any point on these pieces give an additive sharing of the function output. Among many functions, distributed point function (DPF) and distributed comparison function (DCF) are popularly used in the setting where the function is split into two pieces. In this work, we use DCF to perform secure comparisons. Below we describe more on DCFs.

Distributed comparison function (DCF) [BCG⁺21] is a comparison function $f_{\alpha, \beta}^<$ outputting β if input $x \leq \alpha$ and zero otherwise. In the context of the current work, we consider a secure 2-party DCF (a rider and a driver) with a semi-honest dealer (SP).

Secure communication channel between a rider and a driver. In prior works, it is assumed that a rider and a driver have a secure point-to-point communication channel. Here, we also rely on the same assumption and can be established in the following way. Let R and D want to establish a secure communication channel. R and D locally generate a public-key secret-key pair, that is, (sk_R, pk_R) and (sk_D, pk_D) respectively. R and D send pk_R and pk_D to SP respectively and SP forwards pk_R to D and pk_D to R. Note that SP is a semi-honest adversary and does not resort to man-in-the-middle attacks. Now, R and D can securely interact by encrypting their secret messages using respective public keys.

Key setup and establishing a common key. Given a secure point-to-point communication channel, it is straightforward to establish a common secret key. Below, we describe how it can be achieved. Let us consider R and D want to obtain a common secret key k_{RD} . R and D locally sample keys k_R and k_D respectively. Now, they exchange the keys using the secure communication channel, that is, R computes $c_R^{(D)} = \text{Enc}(pk_D, k_R)$ and D computes $c_D^{(R)} = \text{Enc}(pk_R, k_D)$. R and D exchange the ciphertexts via SP. R decrypts and gets $k_D = \text{Dec}(sk_R, c_D^{(R)})$, similarly, D obtains $k_R = \text{Dec}(sk_D, c_R^{(D)})$. Finally, R and D output $k_{RD} = k_R + k_D$. We formally describe the key setup protocol (Π_{KeySetup}) in Fig. 12. Thereafter, for all secure communications, R and D can use the common key k_{RD} to evaluate a PRF on a common value, which can be used as the key for the one-time pad.

Protocol $\Pi_{\text{KeySetup}}(\mathcal{R}, \mathcal{D}, \text{SP})$

Protocol:

- $\forall R \in \mathcal{R}, D \in \mathcal{D}$, sample k_R and k_D respectively.
- R and D communicate the respective keys via secure point-to-point communication channel. That is,
 - R and D compute $c_R^{(D)} = \text{Enc}(pk_D, k_R)$ and $c_D^{(R)} = \text{Enc}(pk_R, k_D)$ respectively.
 - R sends $c_R^{(D)}$ and D sends $c_D^{(R)}$ to SP.
 - SP forwards $c_D^{(R)}$ to R and $c_R^{(D)}$ to D.

- R gets $k_D = \text{Dec}(\text{sk}_R, c_D^{(R)})$ and D gets $k_R = \text{Dec}(\text{sk}_D, c_R^{(D)})$.
- Finally, R and D output $k_{RD} = k_R + k_D$.

Figure 12: Common key setup

C Additional Benchmarking Data

C.1 Comparison of Multi Rider-Driver Matching with Prior Works

We emphasize that prior works on the area of ride-sharing services limit their discussion on oblivious matching between a single rider and a single driver. Most of the works ensure at least one party (say, rider) learns whether it is matched with another party (a driver). If the riders are allowed to make the choice of the driver, this may lead to starvation for a set of users (riders/drivers), which would result in a slow system as discussed in Section 5. This work addresses the question of obtaining optimal matching simultaneously. Furthermore, due to the choice of the system of model of this work, we obtain the optimal matching without any additional interaction, whereas, prior works would require to perform an additional MPC among all the riders and drivers to obtain the optimal matching, which has significant overhead. Although TOPPool [PGT⁺19] addresses only the single rider-driver matching case, we analyze the multi rider-driver scenario for both intersection-based and end-point based matching algorithms by implementing the method discussed in *Attempt 2* in Section 1, with an additional $O(N)$ overhead for parallel matching. Note that this is a significantly simplified approach compared to using MPC, designed just for comparison purpose and may not yield an optimal matching, as described in Section 1. Implementing MPC for TOPPool to achieve optimal matching is beyond the scope of our work.

Fig. 13 provides the comparison of communication as well as runtime with the intersection based matching protocol of TOPPool. We consider an equal number of riders and drivers, varying both from 1 to 100. Additionally, we vary each user's route length from 32 to 4096. We observe that, our work requires $\sim 8 - 11\times$ less communication, whereas, regarding runtime, our protocol is, on average $1.97\times$ faster than TOPPool in most cases.

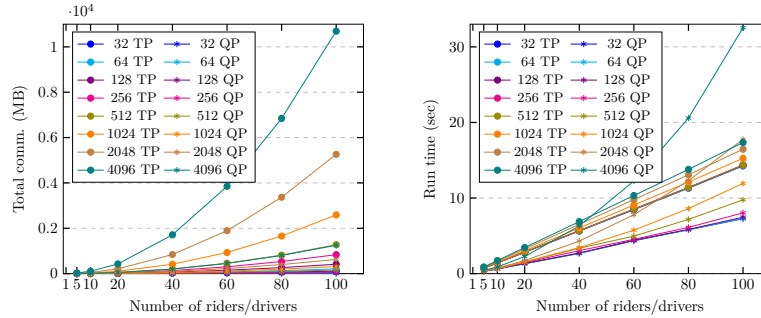


Figure 13: Performance comparison over a MAN network between the intersection based matching protocols of TOPPool (TP) and QuickPool (QP), for varying numbers of riders/drivers in $\{1, 5, 10, 20, 40, 60, 80, 100\}$ and route length in $\{2^j\}_{j \in [5, 12]}$.

Similarly, in Fig. 14 we provide the comparison of communication as well as runtime with the end-point based matching protocol of TOPPool. Here, additionally, we vary the value of $\rho \in \{4, 8, 16\}$ as well, where ρ denotes the maximum threshold for start and end distances. We observe that our protocol requires $\sim 4.5 - 48\times$ less communication than TOPPool.

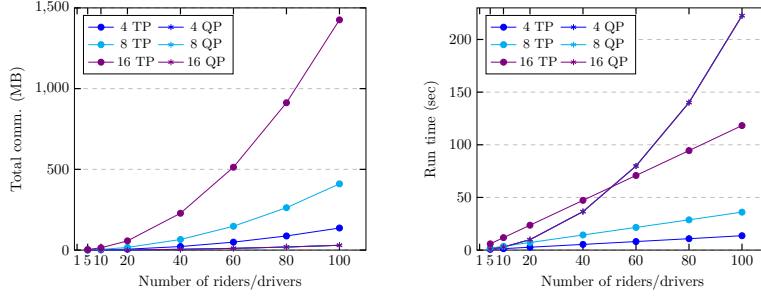


Figure 14: Performance comparison over a MAN network between the end-point based matching protocols of TOPPool (TP) and QuickPool (QP), for varying numbers of riders/drivers in $\{1, 5, 10, 20, 40, 60, 80, 100\}$ and maximum threshold in $\{4, 8, 16\}$.

C.2 Multi-threading at SP

In this work, we consider a service provider (SP) with significantly more computational resources than the riders and the drivers. Thus, in our implementation, we consider SP parallelizing its operations by utilizing multiple threads. We execute our intersection based matching protocol, with 10, 20, 40, 64, and 80 threads and we observe that our protocol achieves the best runtime with 64 threads. Table 4 demonstrates the runtime considering different number of threads and varying number of riders and drivers. Here, we consider each rider and driver has a route of length 2048.

Table 4: Runtime (sec) of intersection based matching algorithm over a MAN network, considering various number of threads used by SP with route length = 2048, for varying numbers of riders/drivers in $\{1, 5, 10, 20, 40, 60, 80, 100\}$.

# threads	# riders/ drivers							
	1	5	10	20	40	60	80	100
1	0.08	0.81	2.66	6.41	17.31	33.03	40.61	55.89
10	0.10	0.43	1.01	2.70	7.73	15.30	24.83	37.34
20	0.10	0.41	0.87	2.20	5.91	11.35	18.49	27.53
40	0.09	0.40	0.82	2.07	5.55	10.62	17.28	25.64
64	0.08	0.36	0.74	1.73	4.31	7.79	12.29	17.77
80	0.09	0.41	0.82	2.05	5.55	10.55	17.15	25.33

We also test our end-point based matching algorithm by allowing SP to utilize a higher number of threads for different rider and driver counts. However, we observe no change in runtime, as the bottleneck lies on the rider or driver side. Since these are mobile low-end devices, they lack the computational capacity to leverage multi-threading.